Học viện
Công nghệ Bưu chính Viễn thông

KHOA TRÍ TUỆ NHÂN TẠO
Faculty of Artificial Intelligence

Bài 6.3: Transformer

Giảng viên: ThS. Võ Ngọc Bích Uyên

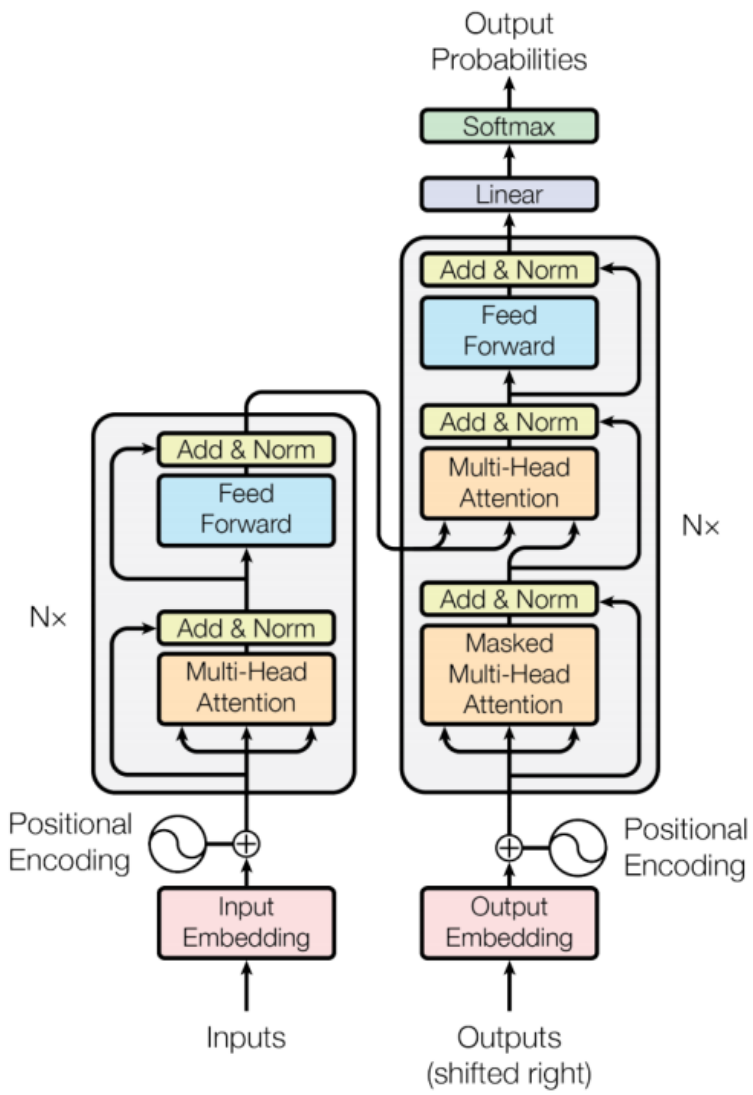
Nội dung

- Tổng quan về Transformer
- Từ mạng RNN đến các mô hình NLP dựa trên sự chú ý
- Mô hình Transformer
- Nhược điểm và các biến thể của transformer

Tổng quan về Transformer

Tổng quan về Transformer

- Sequence-to-sequence (Chuỗi sang chuỗi).
- Bộ mã hóa - Bộ giải mã (Encoder-Decoder).
- Tác vụ: Dịch máy với ngữ liệu song ngữ (parallel corpus).
- Dự đoán từng từ được dịch.
- Hàm chi phí/sai số cuối cùng là cross-entropy error nằm trên một bộ phân loại softmax.



Transformers: Is attention all we need?

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Transformers đã tạo ra một cuộc cách mạng trong lĩnh vực NLP

Courtesy of Paramount Pictures

Kết quả tuyệt vời với Transformers: Machine Translation

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

Kết quả tuyệt vời với Transformers: Document Generation

Model	Test perplexity	ROUGE-L
seq2seq-attention, $L = 500$	5.04952	12.7
Transformer-ED, $L = 500$	2.46645	34.2
Transformer-D, $L = 4000$	2.22216	33.6
Transformer-DMCA, no MoE-layer, $L = 11000$	2.05159	36.2
Transformer-DMCA, MoE-128, $L = 11000$	1.92871	37.9
Transformer-DMCA, MoE-256, $L = 7500$	1.90325	38.8

The old standard

Transformers all the way down.

Kết quả tuyệt vời với Transformers: SuperGLUE

SuperGLUE là một bộ các tác vụ NLP đầy thách thức, bao gồm trả lời câu hỏi, khử nhập nhằng nghĩa của từ, giải quyết đồng tham chiếu và suy luận ngôn ngữ tự nhiên.

Rank	Name	Model	URL	Score	BoolQ	CB	COPA	MultiRC	ReCoRD	RTE	WiC	WSC	AX-b	AX-g
1	JDExplore d-team	Vega v2		91.3	90.5	98.6/99.2	99.4	88.2/62.4	94.4/93.9	96.0	77.4	98.6	-0.4	100.0/50.0
+ 2	Liam Fedus	ST-MoE-32B		91.2	92.4	96.9/98.0	99.2	89.6/65.8	95.1/94.4	93.5	77.7	96.6	72.3	96.1/94.1
3	Microsoft Alexander v-team	Turing NLR v5		90.9	92.0	95.9/97.6	98.2	88.4/63.0	96.4/95.9	94.1	77.1	97.3	67.8	93.3/95.5
4	ERNIE Team - Baidu	ERNIE 3.0		90.6	91.0	98.6/99.2	97.4	88.6/63.2	94.7/94.2	92.6	77.4	97.3	68.6	92.7/94.7
5	Yi Tay	PaLM 540B		90.4	91.9	94.4/96.0	99.0	88.7/63.6	94.2/93.3	94.1	77.4	95.9	72.9	95.5/90.4
+ 6	Zirui Wang	T5 + UDG, Single Model (Google Brain)		90.4	91.4	95.8/97.6	98.0	88.3/63.0	94.2/93.5	93.0	77.9	96.6	69.1	92.7/91.9
+ 7	DeBERTa Team - Microsoft	DeBERTa / TuringNLRv4		90.3	90.4	95.7/97.6	98.4	88.2/63.7	94.5/94.1	93.2	77.5	95.9	66.7	93.3/93.8
8	SuperGLUE Human Baselines	SuperGLUE Human Baselines		89.8	89.0	95.8/98.9	100.0	81.8/51.9	91.7/91.3	93.6	80.0	100.0	76.6	99.3/99.7
+ 9	T5 Team - Google	T5		89.3	91.2	93.9/96.8	94.8	88.1/63.3	94.1/93.4	92.5	76.9	93.8	65.6	92.7/91.9
10	SPoT Team - Google	Frozen T5 1.1 + SPoT		89.2	91.1	95.8/97.6	95.6	87.9/61.9	93.3/92.4	92.9	75.8	93.8	66.9	83.1/82.6

Kết quả tuyệt vời với Transformers: Large Language Models

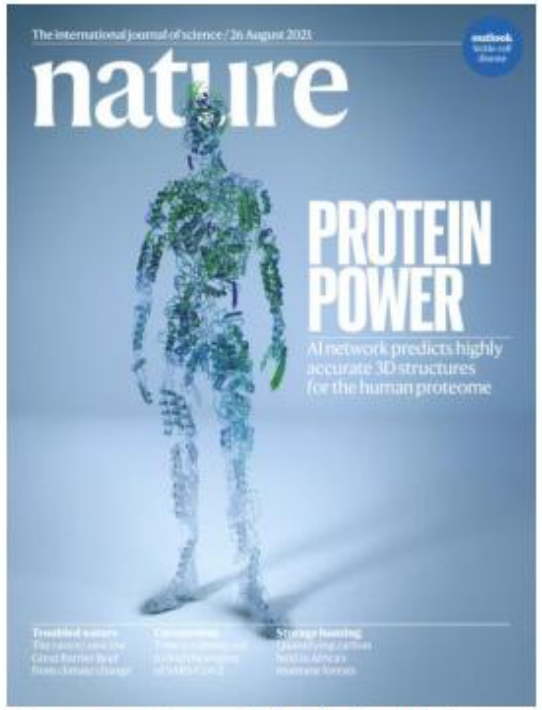
Ngày nay, các mô hình dựa trên Transformer thống trị Bảng xếp hạng Chatbot Arena của LMSYS

Rank* (UB)	Model	Arena Score	95% CI	Votes	Organization	License	Knowledge Cutoff
1	o1-preview	1339	+6/-7	9169	OpenAI	Proprietary	2023/10
1	ChatGPT-4o-latest (2024-09-03)	1337	+4/-4	16685	OpenAI	Proprietary	2023/10
3	o1-mini	1314	+6/-5	9136	OpenAI	Proprietary	2023/10
4	Gemini-1.5-Pro-Exp-0827	1299	+4/-3	31928	Google	Proprietary	2023/11
4	Grok-2-08-13	1293	+4/-3	27731	xAI	Proprietary	2024/3
6	GPT-4o-2024-05-13	1285	+3/-3	93428	OpenAI	Proprietary	2023/10
7	GPT-4o-mini-2024-07-18	1272	+3/-3	33166	OpenAI	Proprietary	2023/10
7	Claude 3.5 Sonnet	1269	+3/-3	67165	Anthropic	Proprietary	2024/4

Gemini (Google) ChatGPT (OpenAI) Claude (Anthropic) Llama (Meta)

Triển vọng ngoài lĩnh vực NLP

Protein Folding



[Jumper et al. 2021] aka AlphaFold2!

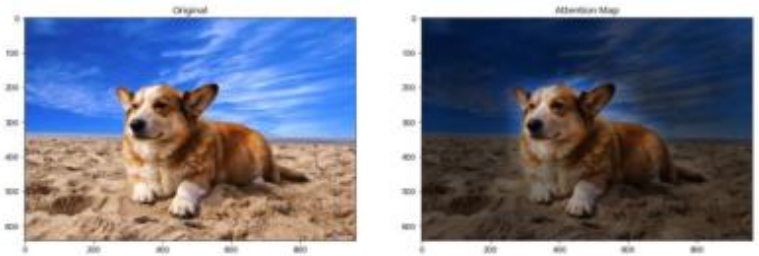
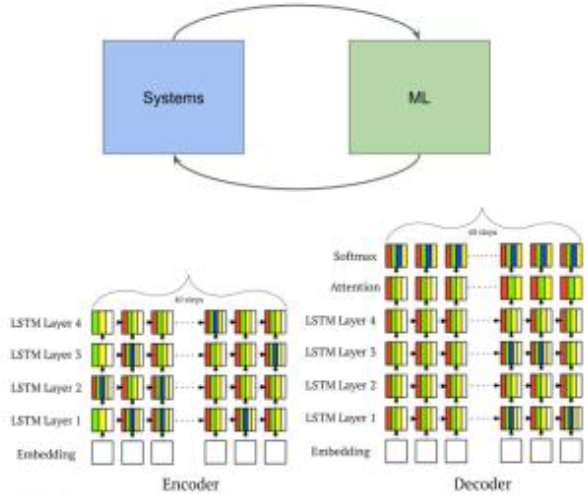


Image Classification

[Dosovitskiy et al. 2020]: Vision Transformer (ViT) outperforms ResNet-based baselines with substantially less compute.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	–
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	–
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	–
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03	–
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k



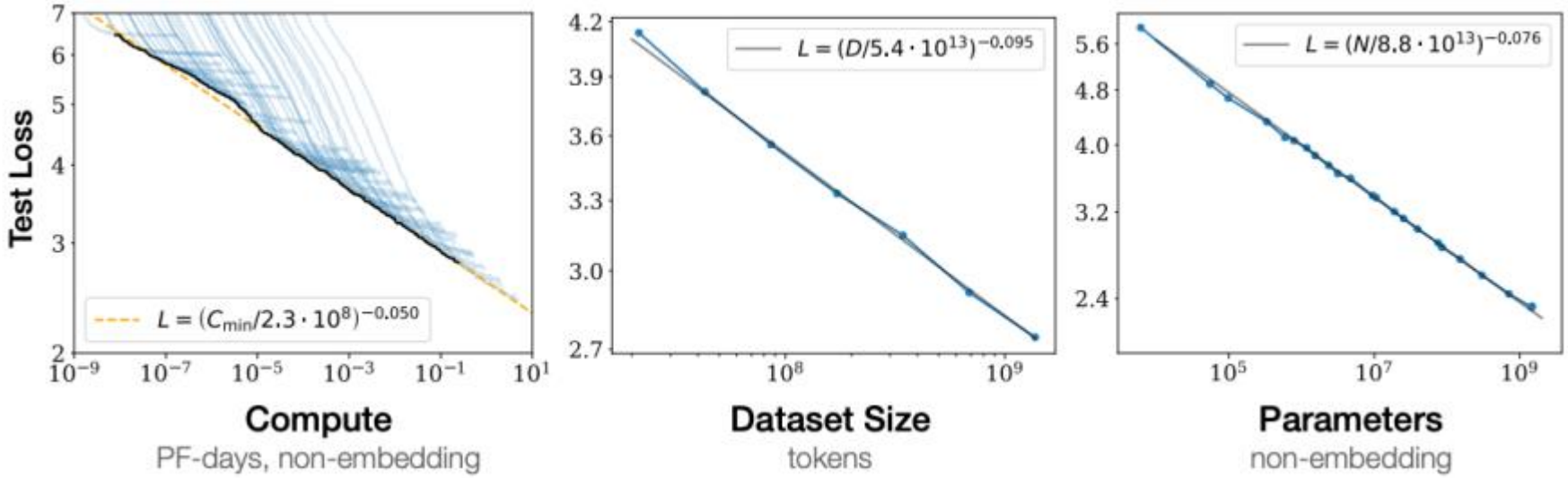
ML for Systems

[Zhou et al. 2020]: A Transformer-based compiler model (GO-one) speeds up a Transformer model!

Model (#devices)	GO-one (s)	HP (s)	METIS (s)	HDP (s)	Run time speed up over HP / HDP	Search speed up over HDP
2-layer RNNLM (2)	0.173	0.192	0.355	0.191	9.9% / 9.4%	2.95x
4-layer RNNLM (4)	0.210	0.239	0.503	0.251	13.8% / 16.3%	1.76x
8-layer RNNLM (8)	0.320	0.332	OOM	0.764	3.8% / 58.1%	27.8x
2-layer GNMT (2)	0.301	0.384	0.344	0.327	27.6% / 14.3%	30x
4-layer GNMT (4)	0.350	0.469	0.466	0.432	34% / 23.4%	58.8x
[illegible]	0.440	0.562	OOM	0.693	21.7% / 36.5%	7.35x
2-layer Transformer-XL (2)	0.223	0.268	0.37	0.262	20.1% / 17.4%	40x
4-layer Transformer-XL (4)	0.290	0.27	OOM	0.259	17.4% / 12.6%	26.7x
8-layer Transformer-XL (8)	0.350	0.46	OOM	0.425	23.9% / 16.7%	16.7x
[illegible]	0.229	0.312	OOM	0.301	26.6% / 23.9%	13.5x
Inception (2) b64	0.423	0.731	OOM	0.498	42.1% / 29.3%	21.0x
AmoebaNet (4)	0.394	0.44	0.426	0.418	26.1% / 6.1%	58.8x
2-stack 18-layer WaveNet (2)	0.317	0.376	OOM	0.354	18.6% / 11.7%	6.67x
4-stack 36-layer WaveNet (4)	0.659	0.988	OOM	0.721	50% / 9.4%	20x
GEOMEAN	-	-	-	-	**20.5% / 18.2%**	**18x**

Scaling Laws: Có phải Transformers là tất cả những gì chúng ta cần?

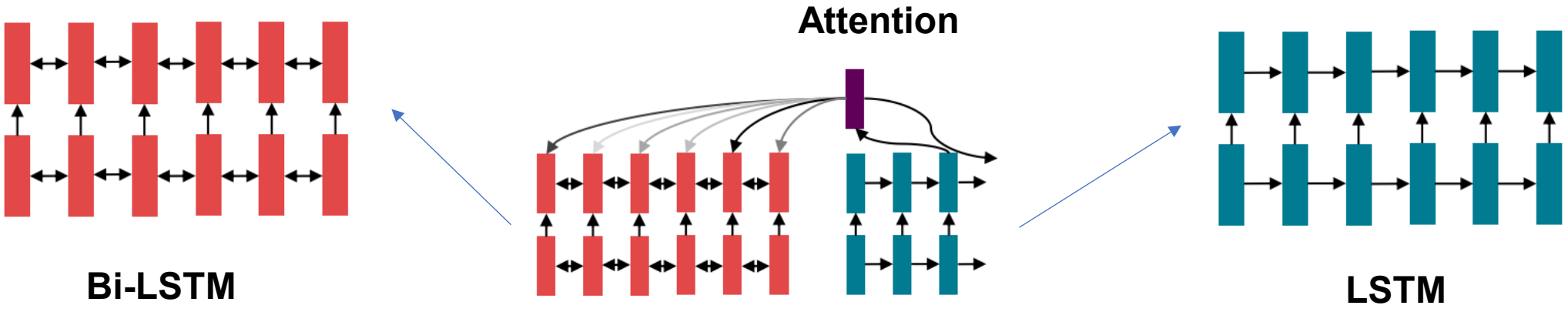
- Với Transformers, hiệu suất mô hình hóa ngôn ngữ cải thiện một cách mượt mà khi chúng ta gia tăng đồng thời kích thước mô hình, dữ liệu huấn luyện và tài nguyên tính toán.
- Mối quan hệ lũy thừa này đã được quan sát qua nhiều cấp độ lớn mà không có dấu hiệu chậm lại!
- Nếu chúng ta tiếp tục mở rộng quy mô các mô hình này (mà không thay đổi kiến trúc), liệu cuối cùng chúng có thể bắt kịp hoặc vượt qua hiệu suất của con người không?



RNNs tới mô hình attention-based

Recurrent models cho (hầu hết) NLP!

- Khoảng năm 2016, chiến lược thực tế trong NLP là mã hóa các câu bằng mạng LSTM hai chiều (ví dụ: câu nguồn trong một bản dịch).
- Xác định đầu ra của bạn (chuỗi phân tích cú pháp, dịch câu, tóm tắt văn bản) dưới dạng một chuỗi và sử dụng mạng LSTM để tạo ra nó.
- Sử dụng sự chú ý (attention) để cho phép truy cập linh hoạt vào bộ nhớ.



Tại sao phải tiến xa hơn Recurrence?

- Giảm thiểu (hoặc ít nhất là không làm tăng) độ phức tạp tính toán trên mỗi lớp.
- Giảm thiểu độ dài đường dẫn giữa bất kỳ cặp từ nào để tạo điều kiện thuận lợi cho việc học các phụ thuộc khoảng cách xa (long-range dependencies).
- Tối đa hóa lượng tính toán có thể được song song hóa.

Động lực cho Transformer: Độ phức tạp tính toán mỗi lớp

Khi độ dài chuỗi (n) << chiều biểu diễn (d), độ phức tạp mỗi lớp của Transformer sẽ thấp hơn so với các mô hình RNN

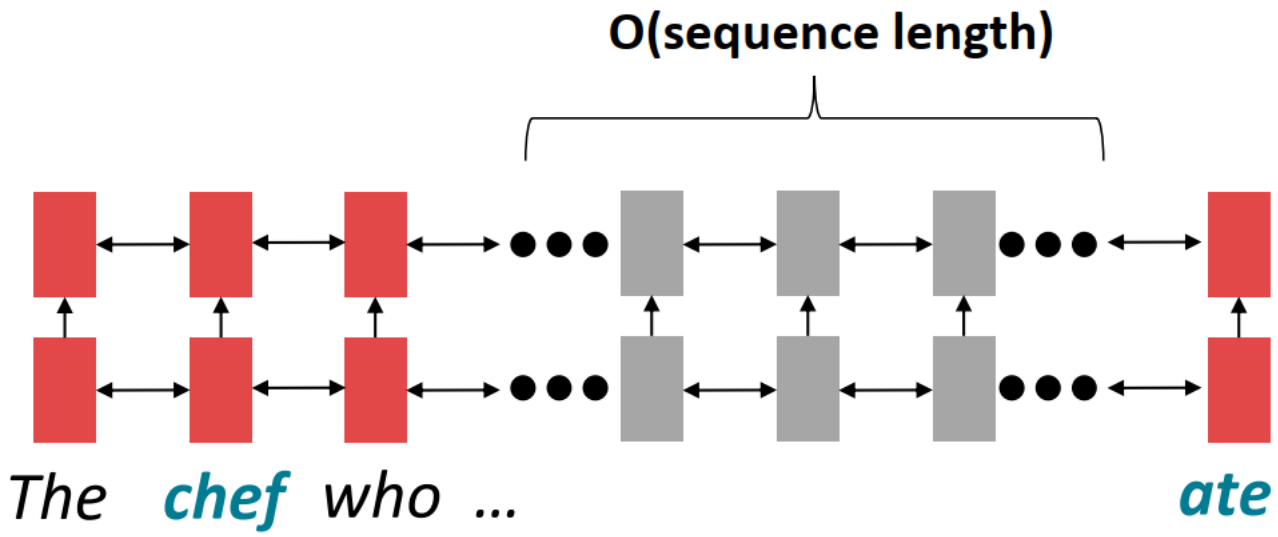
Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Table 1 of the Transformer paper.

Động lực cho Transformer: Giảm thiểu khoảng cách tương tác Tuyến tính

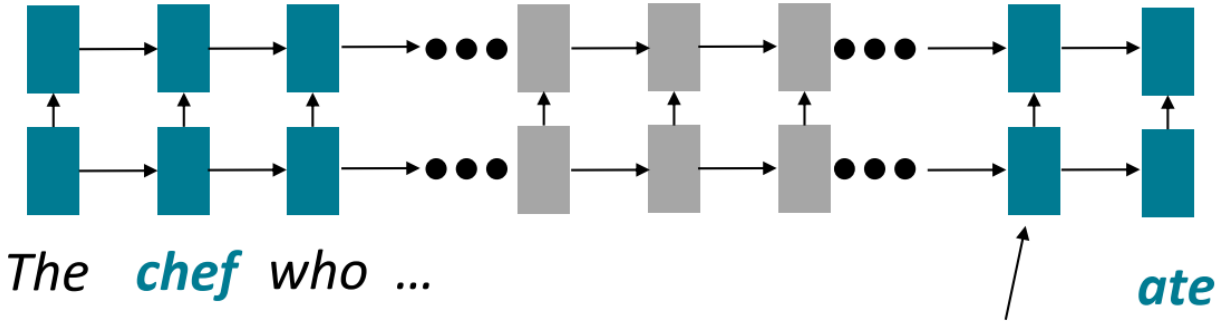
- Mạng unrolled RNN từ "trái sang phải", mã hóa tính cục bộ tuyến tính. Đây là một phương pháp suy nghiệm hữu ích vì các từ ở gần thường ảnh hưởng đến ý nghĩa của nhau.
- Tuy nhiên, vấn đề là RNN mất O(sequence length) bước để các cặp từ ở xa có thể tương tác với nhau. Thông tin của một từ phải đi qua số lớp bằng với độ dài chuỗi.



Động lực cho Transformer:
Giảm thiểu khoảng cách tương tác Tuyến tính

Hậu quả:

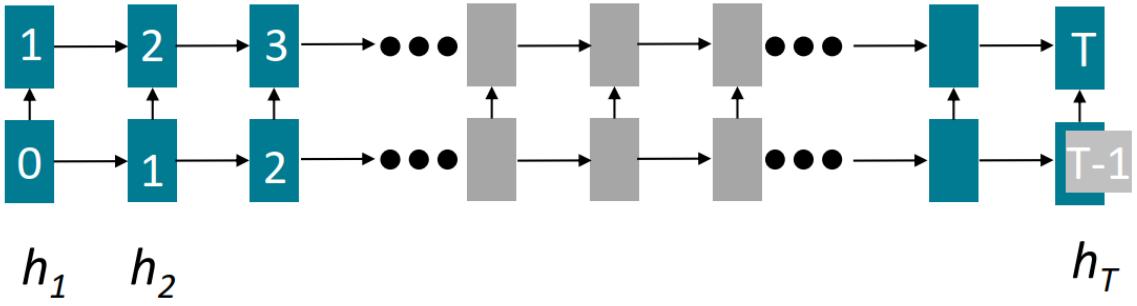
- Khó học các phụ thuộc xa (long-distance dependencies): Gradient phải truyền qua quá nhiều bước thời gian, dẫn đến các vấn đề về biến mất hoặc bùng nổ gradient.
- Thứ tự tuyến tính bị "áp đặt mặc định" (baked in): Kiến trúc này giả định các từ được xử lý theo thứ tự từ trái sang phải. Tuy nhiên, cấu trúc tuần tự đơn thuần không phản ánh được toàn bộ bản chất của ngôn ngữ (vốn có tính phân cấp và các mối quan hệ ngữ pháp phức tạp hơn).



Info of ***chef*** has gone through O(sequence length) many layers!

Động lực cho Transformer: Tối đa hóa khả năng song song hóa

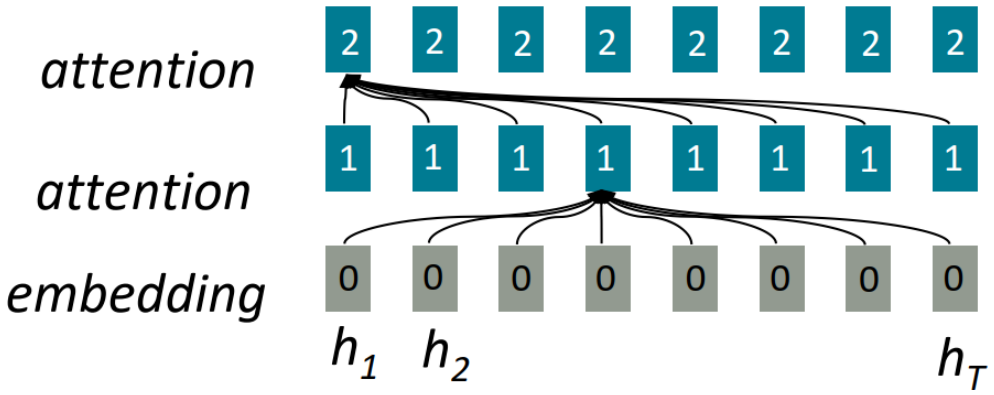
- Quá trình lan truyền tiến (forward) và ngược (backward) trong RNN có O(seq length) các phép toán không thể song song hóa
- GPU (và TPU) có thể thực hiện nhiều tính toán độc lập cùng một lúc. Nhưng các trạng thái ẩn RNN tương lai không thể được tính toán đầy đủ trước khi các trạng thái ẩn RNN quá khứ được tính toán xong.
- Điều này cản trở việc huấn luyện trên các tập dữ liệu rất lớn và đặc biệt có vấn đề khi độ dài chuỗi tăng lên, vì chúng ta không thể nhóm nhiều ví dụ lại với nhau do giới hạn bộ nhớ.



Numbers indicate min # of steps before a state can be computed

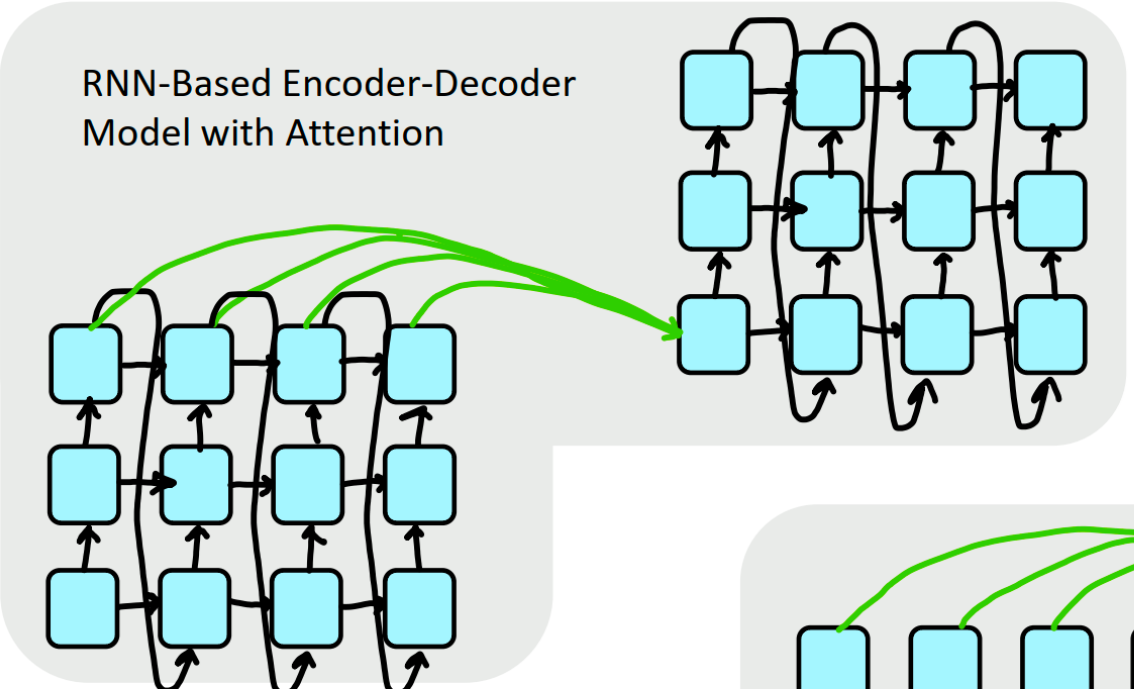
Kiến trúc tổng quan: Transformer hoàn toàn xoay quanh Self-attention

- Nhắc lại: Cơ chế chú ý (attention) coi biểu diễn của mỗi từ như một truy vấn (query) để truy cập và kết hợp thông tin từ một tập hợp các giá trị (values).
- Chúng ta đã học về sự chú ý từ bộ giải mã sang bộ mã hóa trong mô hình sequence-to-sequence hồi quy.
- Sự Tự chú ý (Self-attention) là sự chú ý trong nội bộ encoder-encoder (hoặc decoder-decoder), trong đó mỗi từ chú ý đến mọi từ khác trong cùng phần đầu vào (hoặc đầu ra).



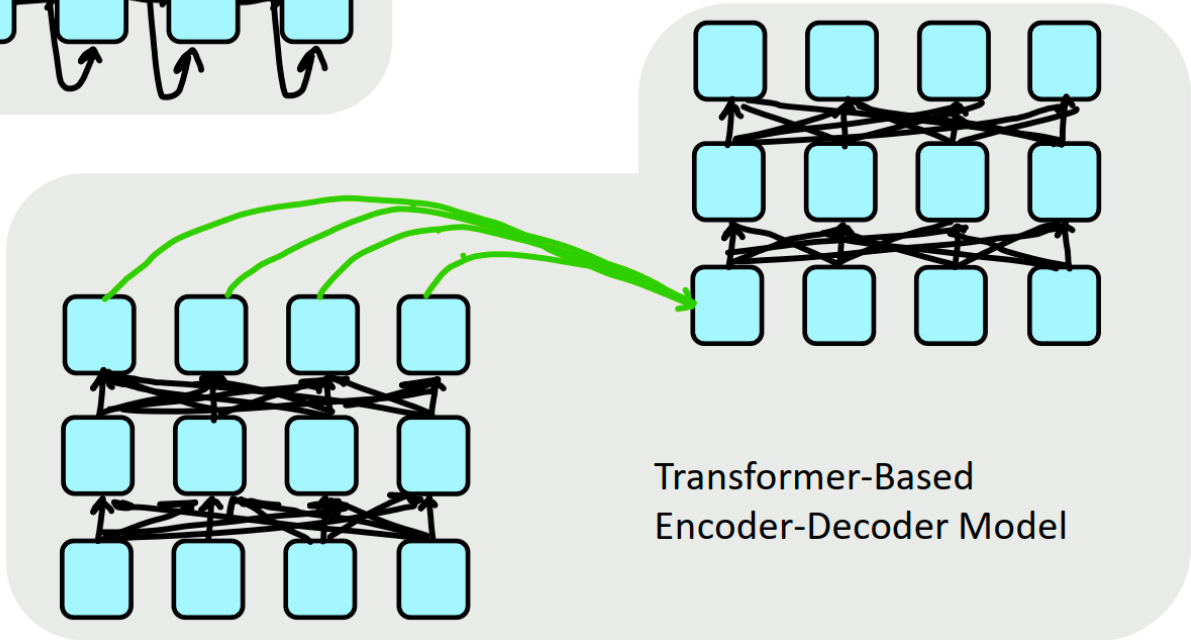
All words attend to all words in previous layer; most arrows here are omitted

Các phụ thuộc tính toán: Recurrence vs. Attention



Transformer Advantages:

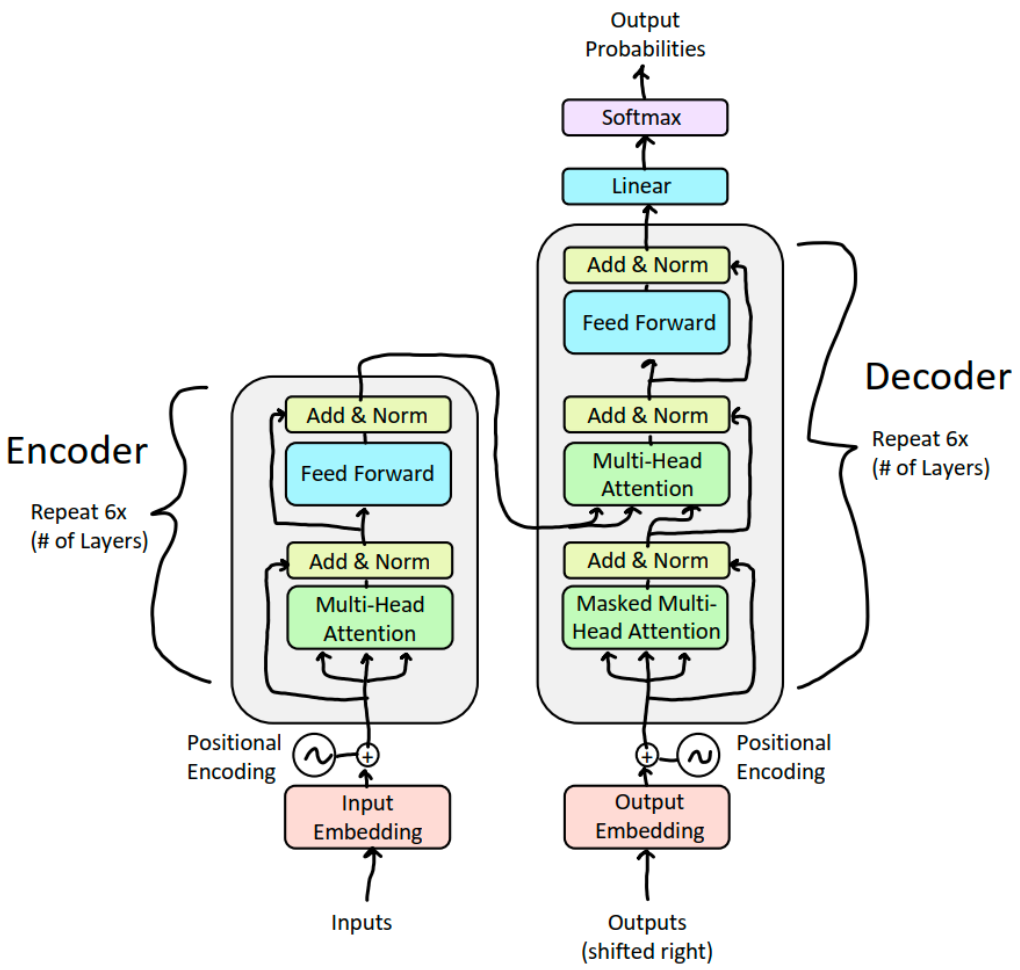
- Number of unparallelizable operations does not increase with sequence length.
- Each "word" interacts with each other, so maximum interaction distance is O(1).



Mô hình Transformer

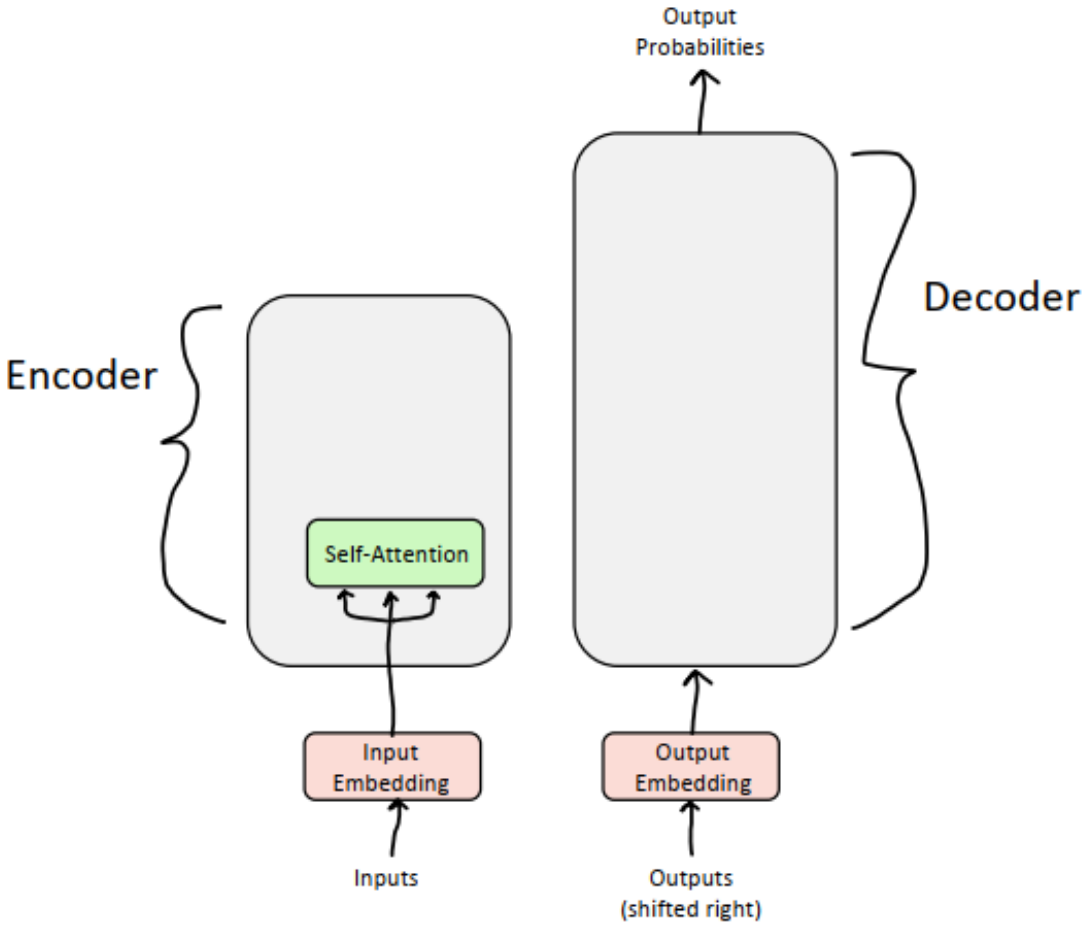
The Transformer Encoder-Decoder

Bao gồm khối Encoder và Decoder, mỗi khối được lặp lại Nx lần (ví dụ: N=6 lớp)



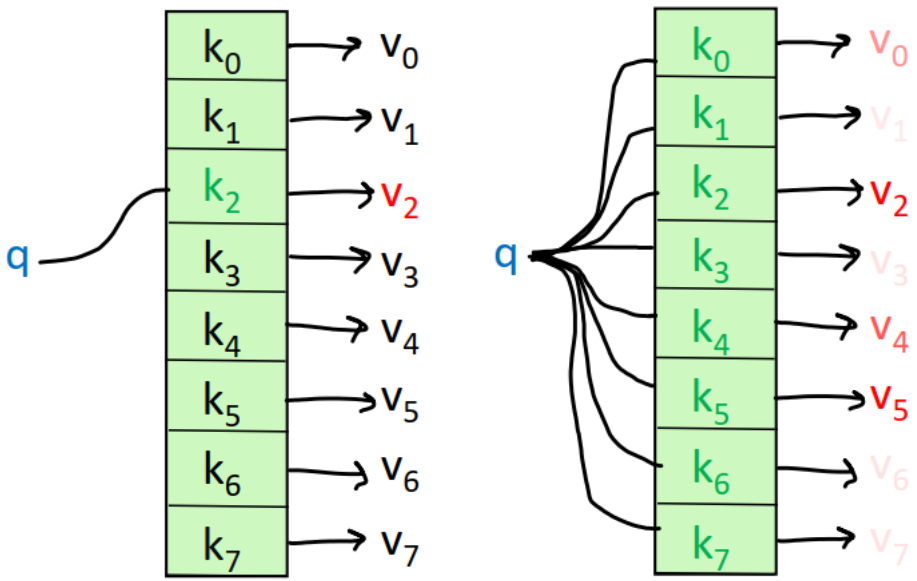
Encoder: Self-Attention

Self-Attention là khối xây dựng cốt lõi của Transformer.



Cơ chế chú ý (Attention Mechanism)

- Hãy coi attention như một bảng băm (hash table) "mờ" hoặc xấp xỉ.
- Để tra cứu một giá trị, chúng ta so sánh một truy vấn (query) với các khóa (keys) trong bảng.
- Trong một bảng băm thông thường: Mỗi truy vấn (mã băm) ánh xạ chính xác tới một cặp khóa-giá trị.
- Trong self-attention: Mỗi truy vấn khớp với mỗi khóa ở các mức độ khác nhau. Chúng ta trả về một tổng các giá trị được tính trọng số bởi mức độ khớp giữa truy vấn-khóa.



Recipe for Self-Attention in the Transformer Encoder

Bước 1: Với mỗi từ x_i, tính toán giá trị query, key và value của nó.

$$q_i = W^Q x_i$$
$$k_i = W^K x_i$$
$$v_i = W^V x_i$$

Bước 2: Tính điểm attention giữa query và keys.

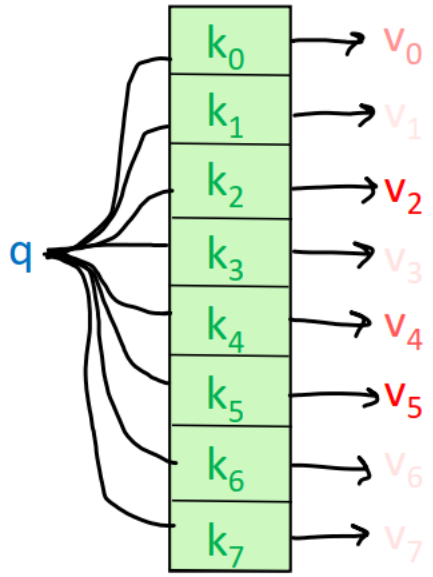
$$e_{ij} = q_i \cdot k_j$$

Bước 3: Áp dụng softmax để chuẩn hóa (normalize) điểm attention.

$$\alpha_{ij} = softmax(e_{ij}) = \frac{exp(e_{ij})}{\sum_k exp(e_{ik})}$$

Bước 4: Tính tổng có trọng số các values.

$$a_i = \sum_j \alpha_{ij} v_j$$



Dạng Vector hóa Self-Attention trong Transformer Encoder

Bước 1: Với các embedding được xếp chồng trong ma trận X, tính **queries**, **keys** và **values**.

$$Q = XW^Q$$
$$K = XW^K$$
$$V = XW^V$$

Bước 2: Tính điểm attention giữa query và keys.

$$E = QK^T$$

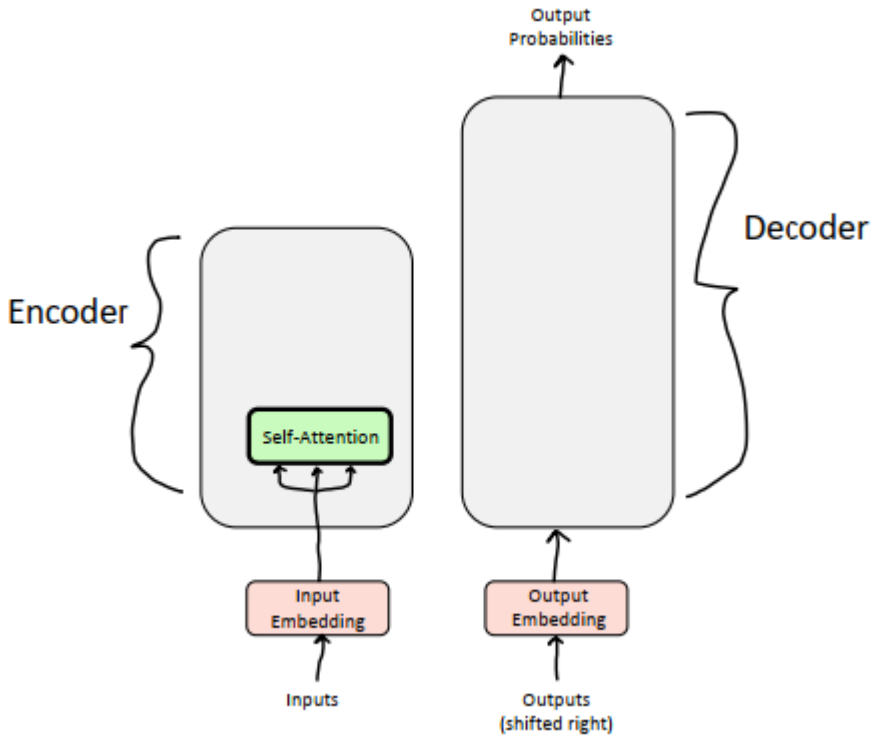
Bước 3: Áp dụng softmax để chuẩn hóa (normalize) điểm attention.

$$A = softmax(E)$$

Bước 4: Tính tổng có trọng số các values.

$$Output = AV = softmax\left(QK^T\right)V$$

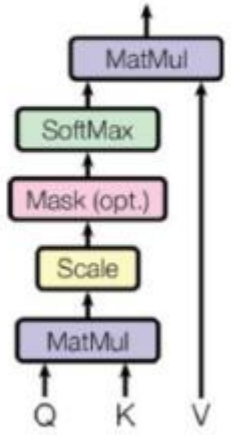
What We Have So Far: (Encoder) Self-Attention!

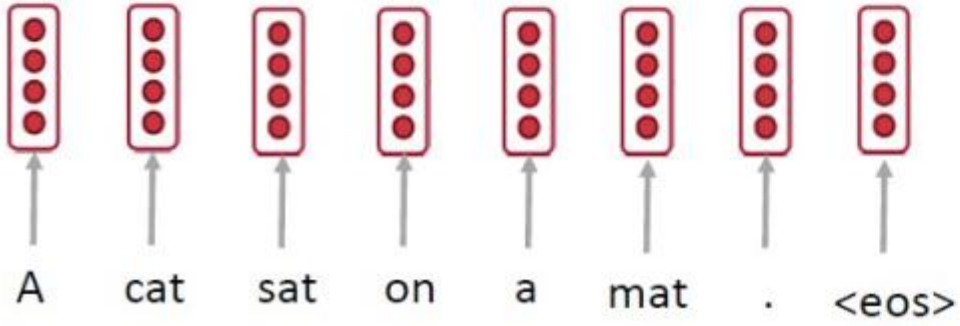


Self-attention: Ví dụ

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

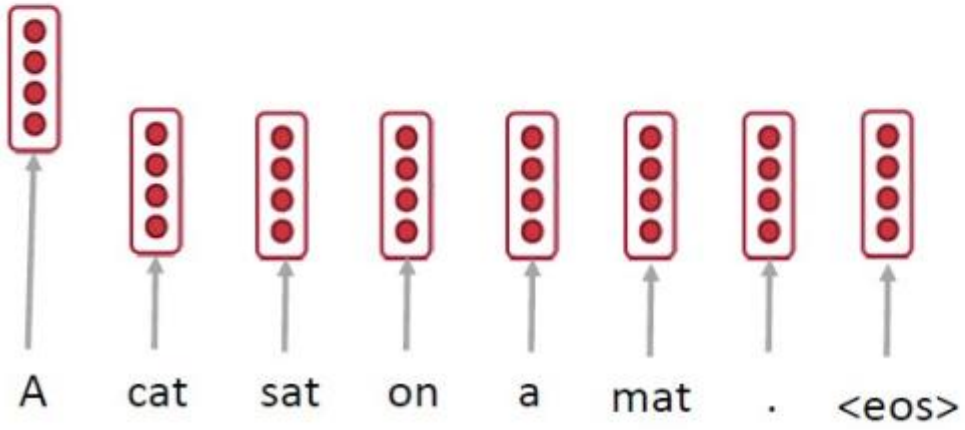
Scaled Dot-Product Attention



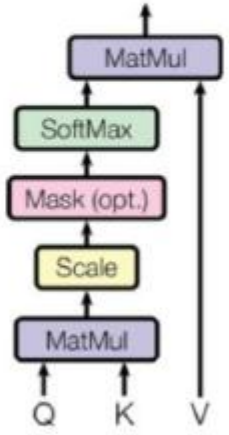


Self-attention: Ví dụ

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

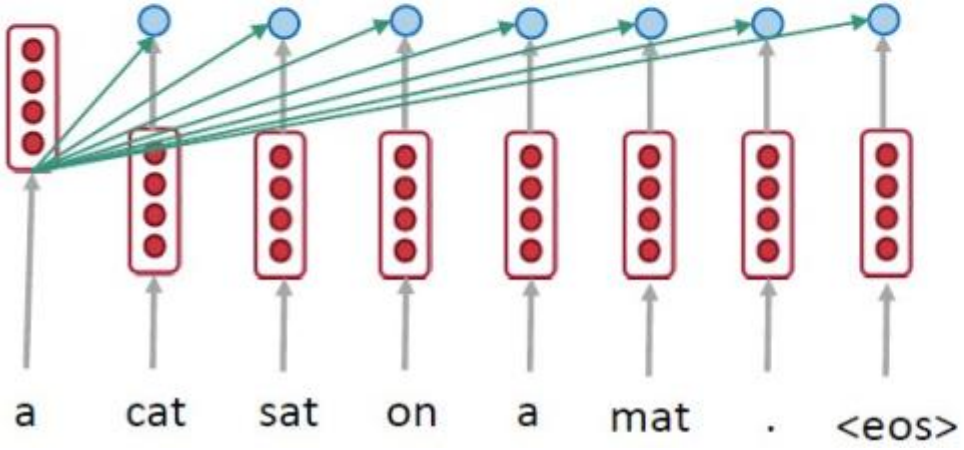


Scaled Dot-Product Attention

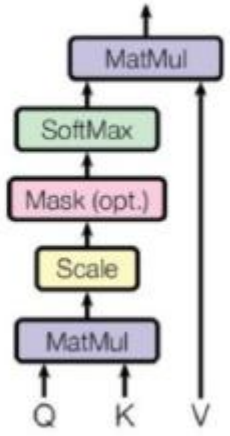


Self-attention: Ví dụ

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

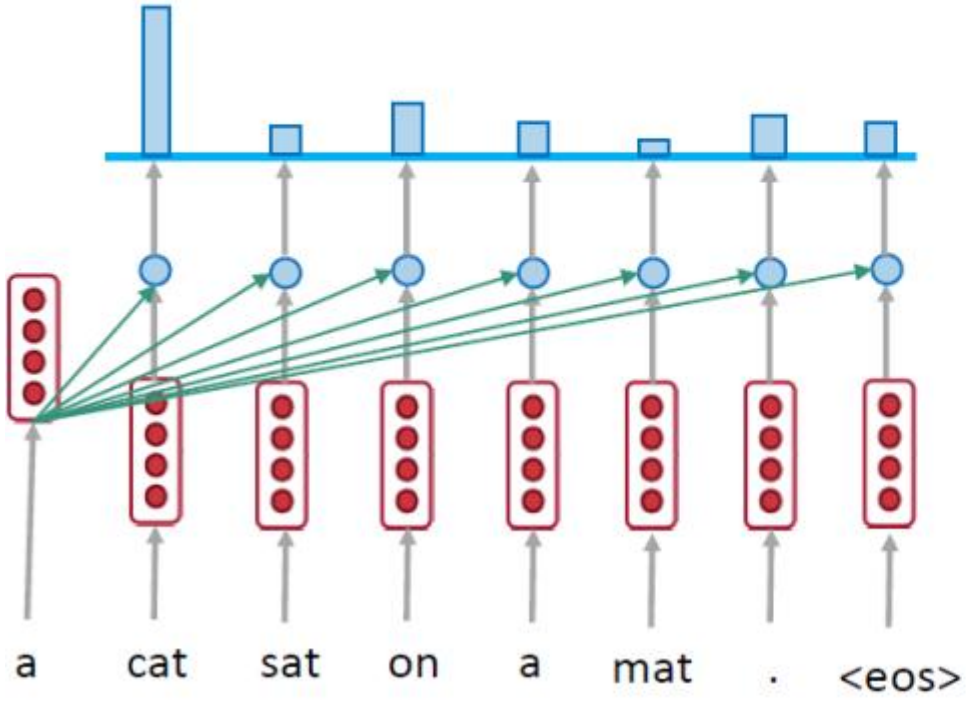


Scaled Dot-Product Attention

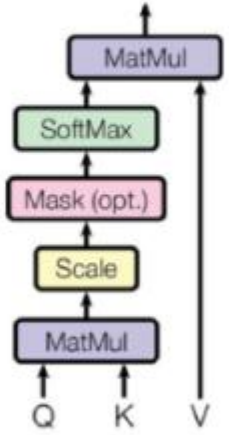


Self-attention: Ví dụ

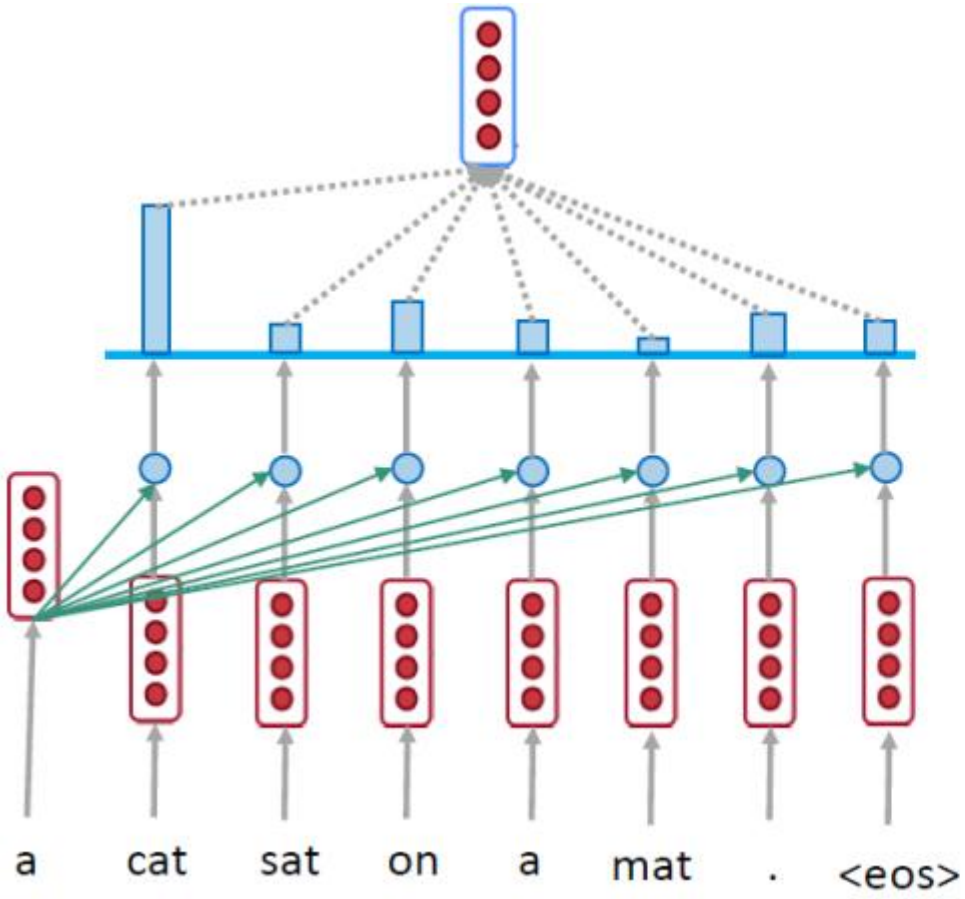
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Scaled Dot-Product Attention

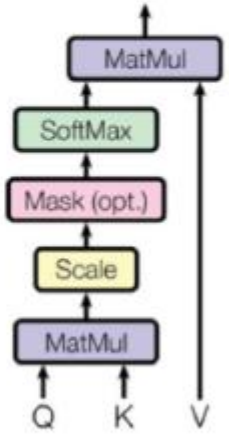


Self-attention: Ví dụ

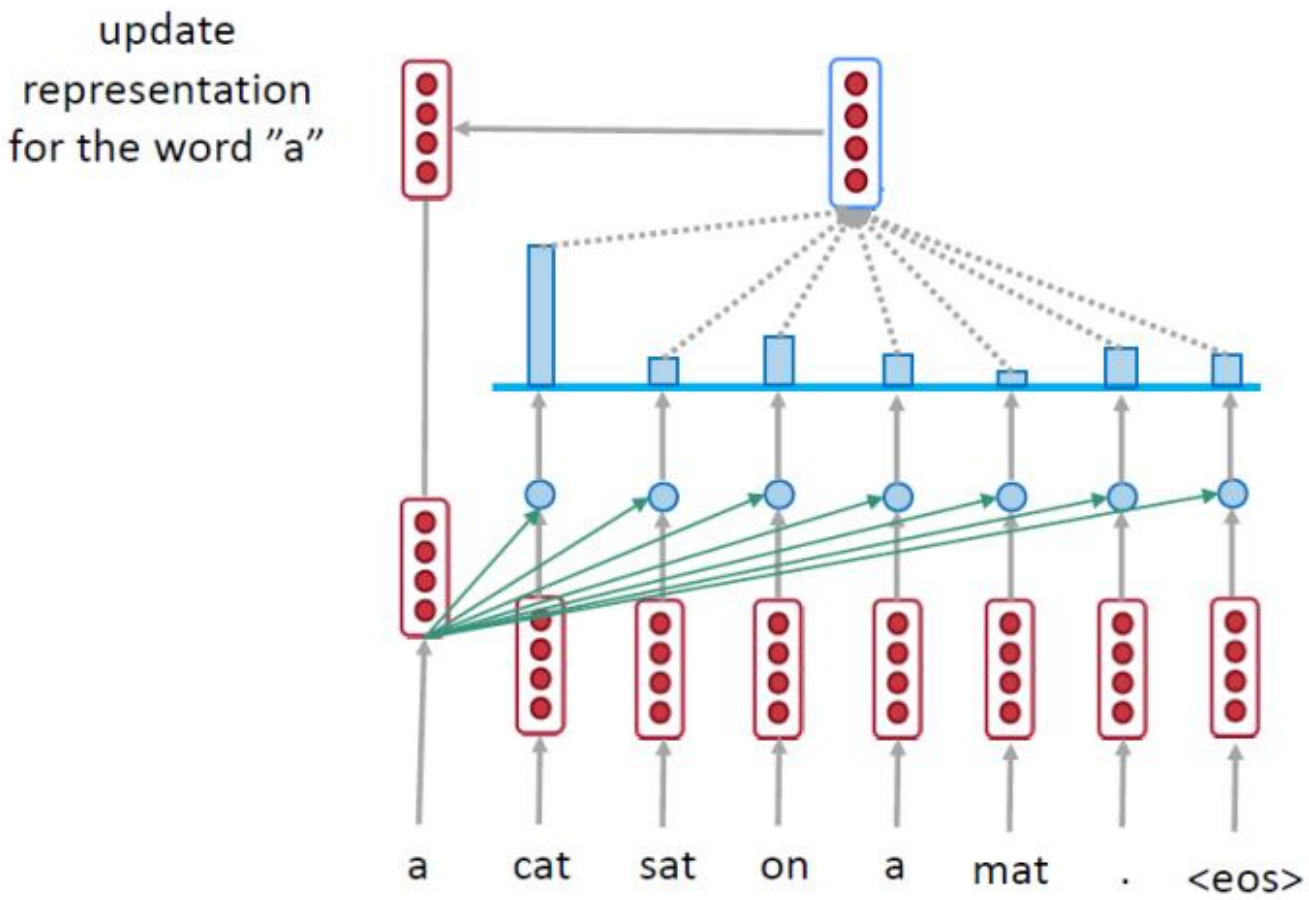


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention

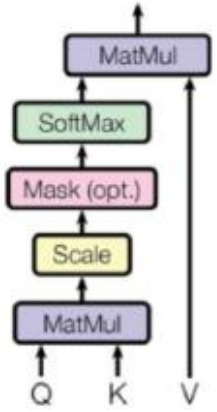


Self-attention: Ví dụ

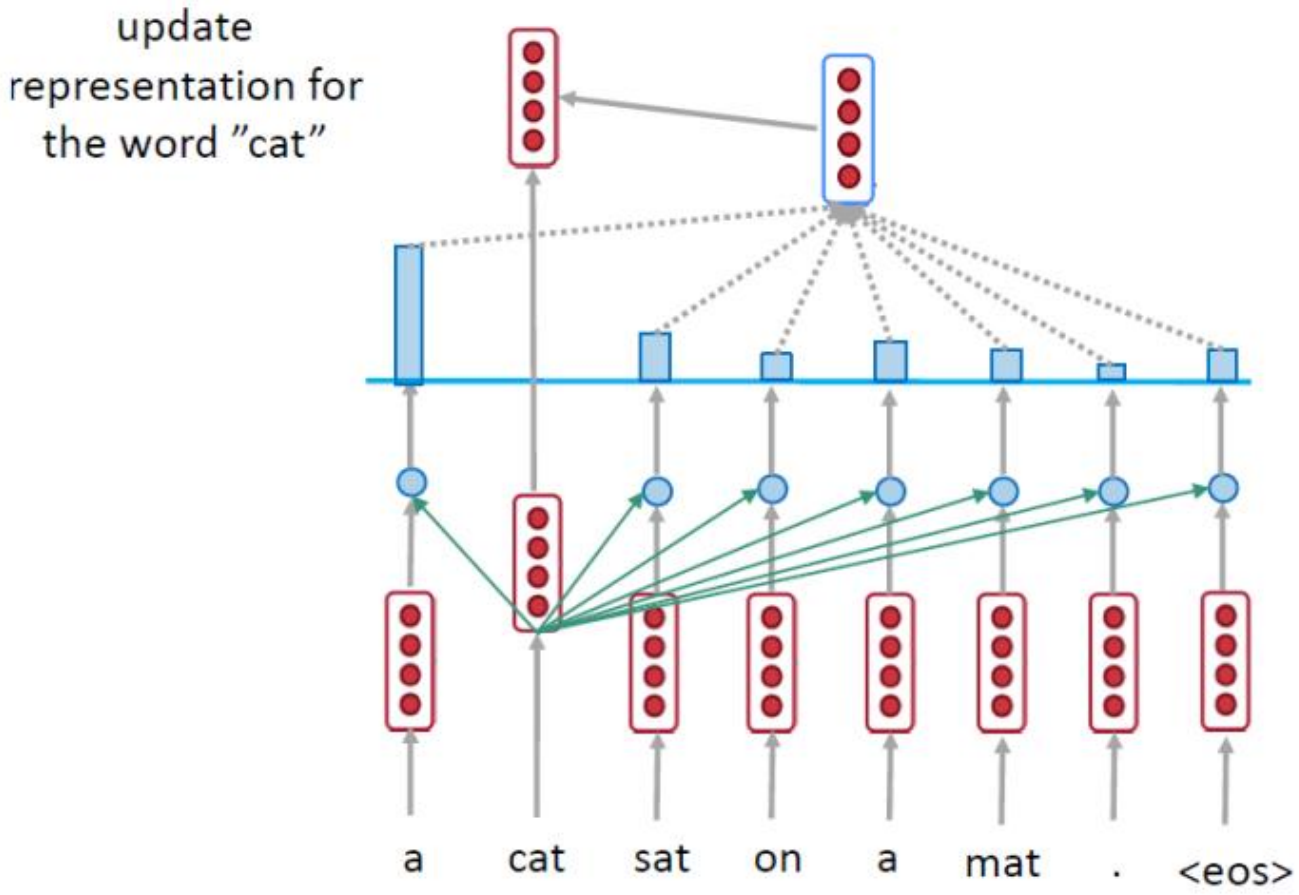


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention

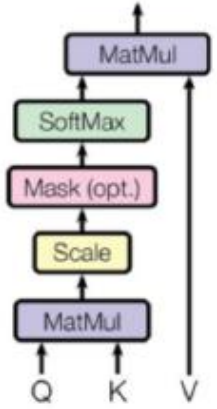


Self-attention: Ví dụ

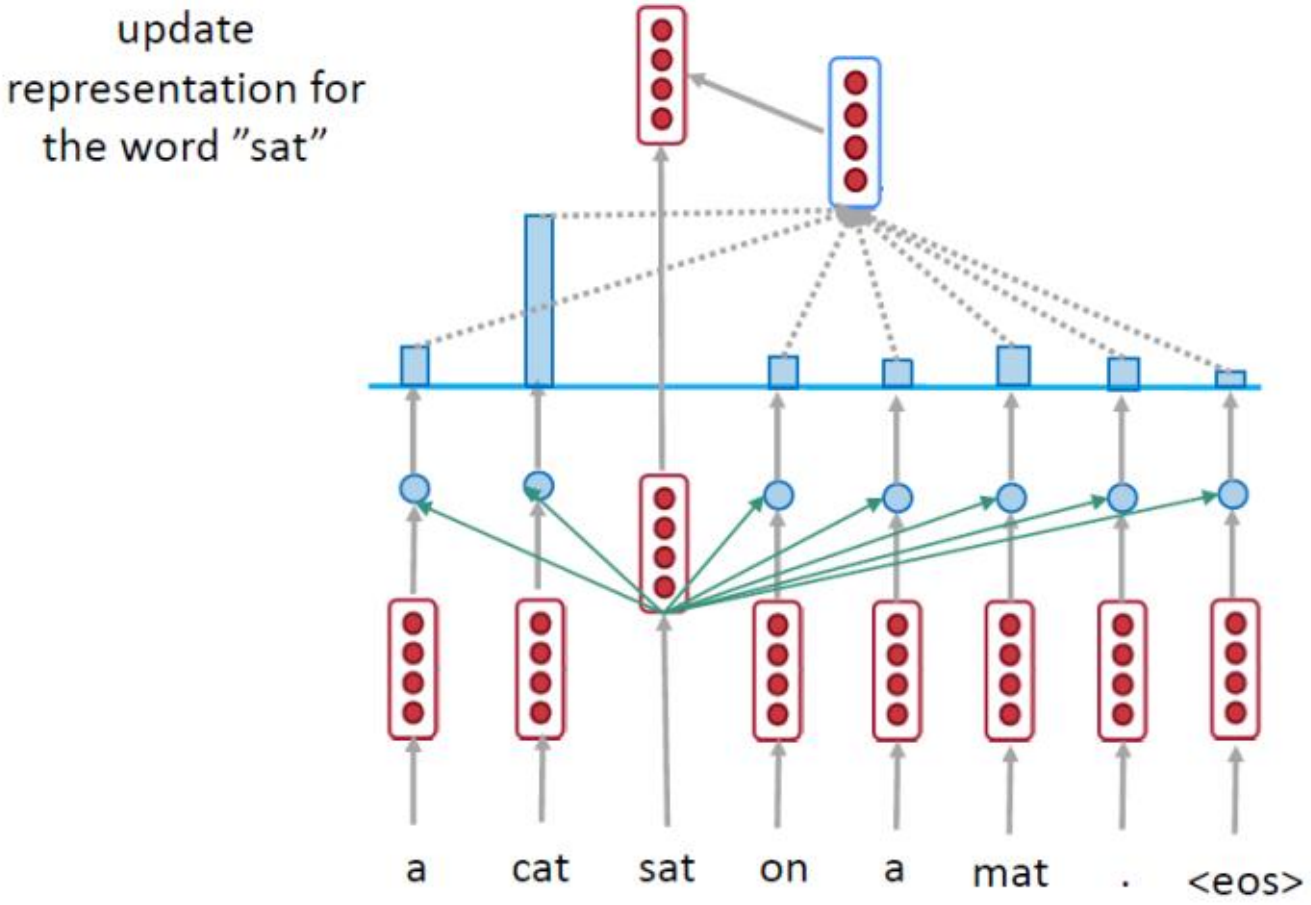


$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Scaled Dot-Product Attention

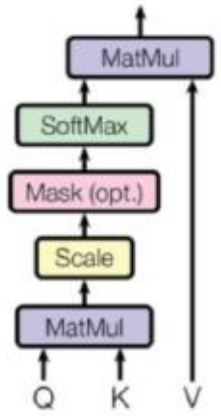


Self-attention: Ví dụ



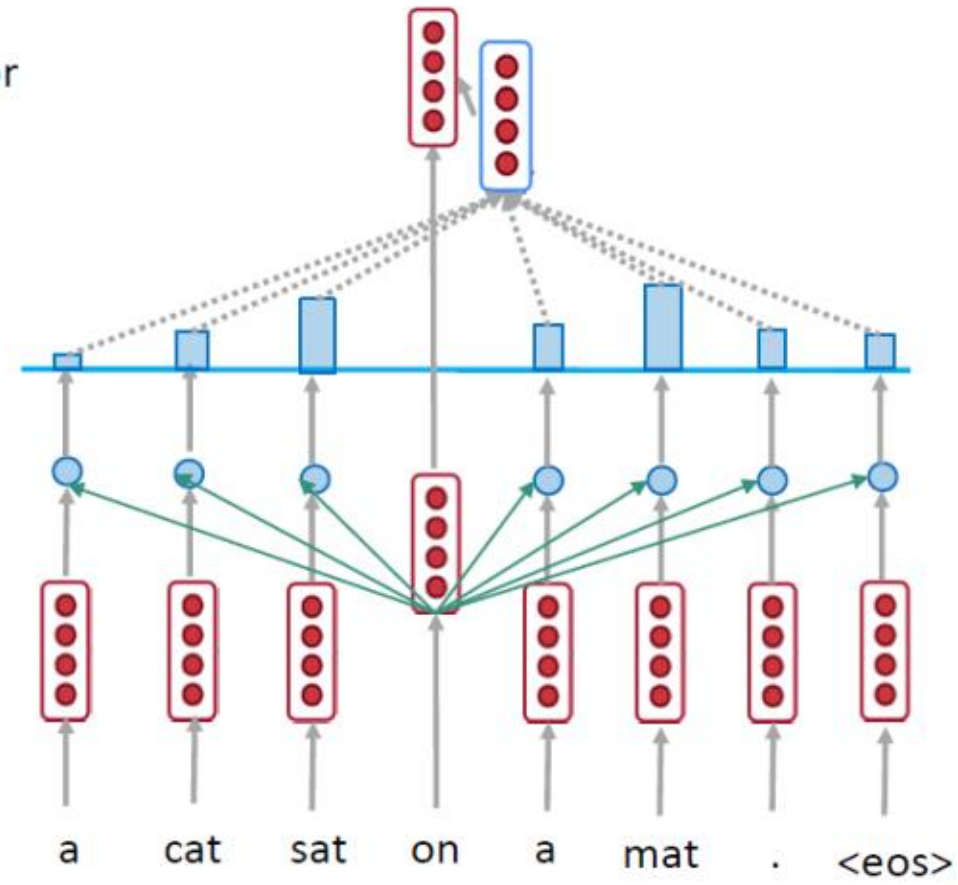
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Scaled Dot-Product Attention



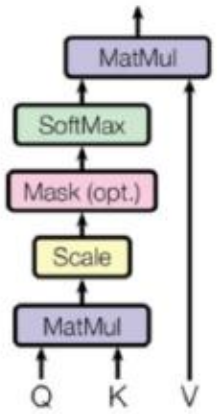
Self-attention: Ví dụ

update representation for the word "on"



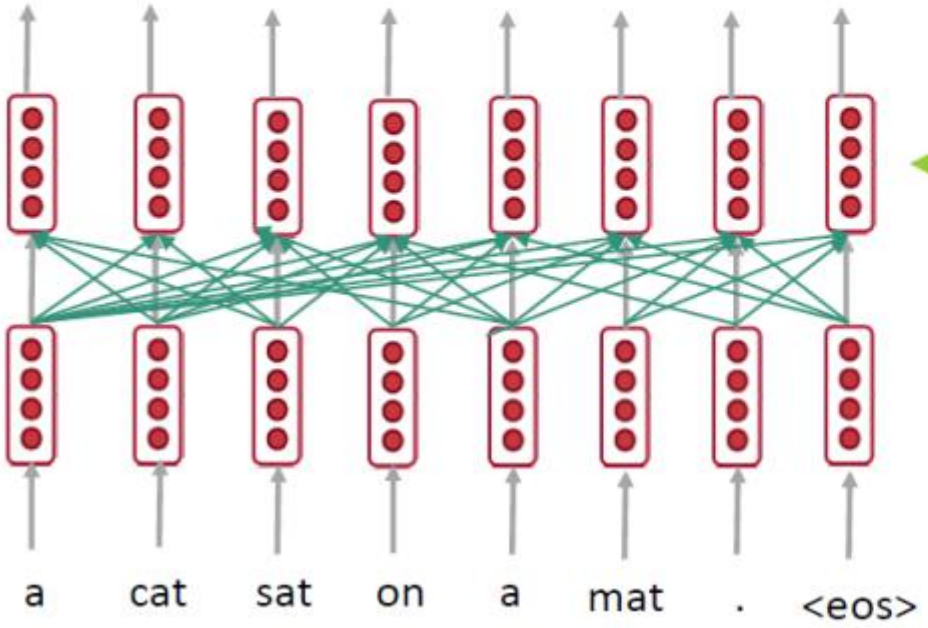
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention



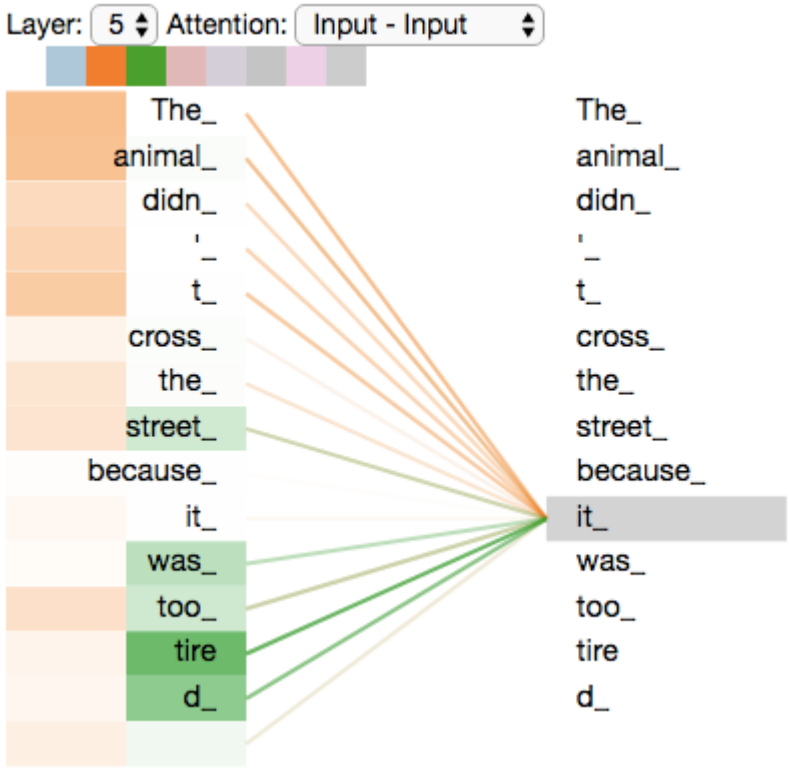
Self-attention: Ví dụ

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



Updated representations
for each word
(one layer)

Trực quan hóa self-attention

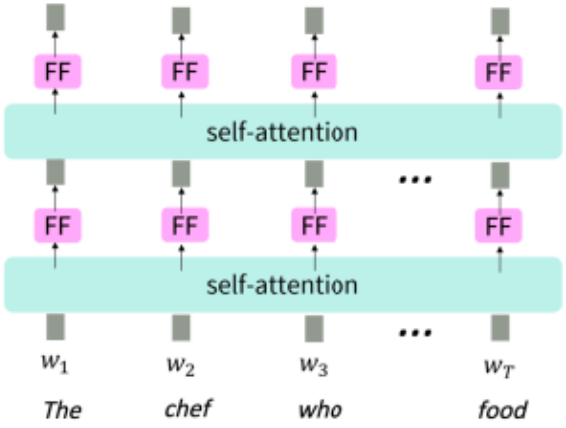


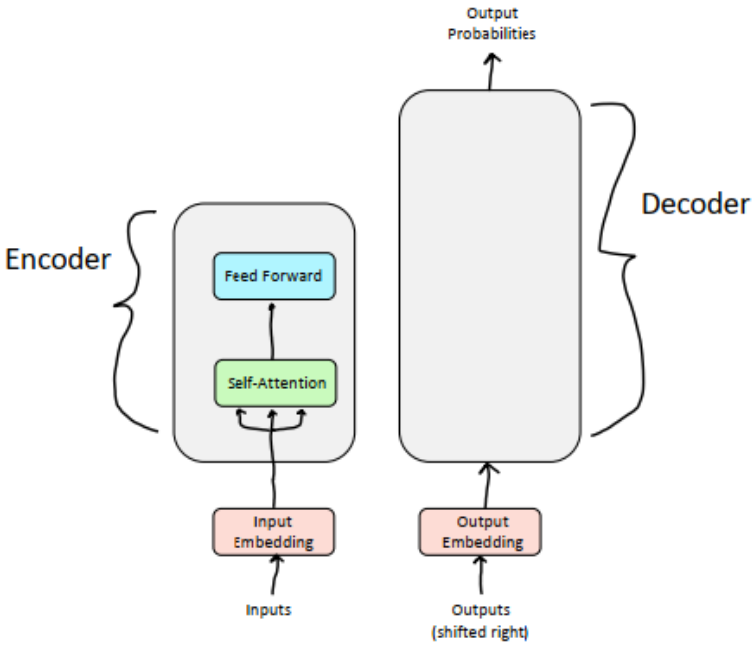
Nhưng attention không hoàn toàn là tất cả!

- Vấn đề: Vì không có các hàm phi tuyến tính theo từng phần tử (element-wise non-linearities), việc tự chú ý chỉ đơn giản là thực hiện việc lấy trung bình lại (reaveraging) các vector giá trị .
- Giải pháp đơn giản: Áp dụng một lớp truyền thẳng (feedforward layer) vào đầu ra của attention, cung cấp sự kích hoạt phi tuyến tính (và bổ sung thêm sức mạnh biểu diễn).

Equation for Feed Forward Layer

$$\begin{aligned} m_i &= MLP(\text{output}_i) \\ &= W_2 * \text{ReLU}(W_1 \times \text{output}_i + b_1) + b_2 \end{aligned}$$



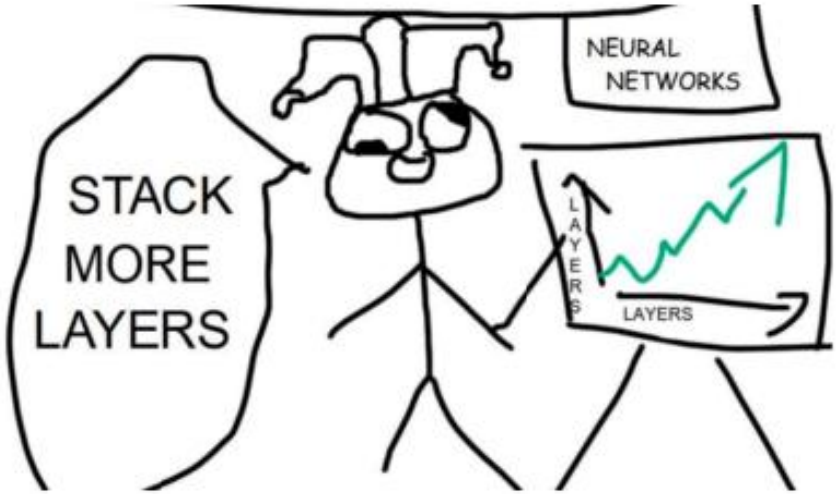


Làm thế nào để điều này hoạt động cho deep networks?

Training Trick #1: Residual Connections

Training Trick #2: LayerNorm

Training Trick #3: Scaled Dot Product Attention



Training Trick #1: Residual Connections

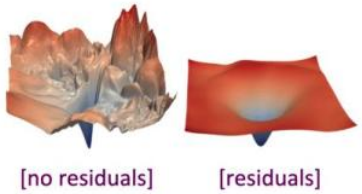
Đây là một kỹ thuật đơn giản nhưng mạnh mẽ từ thị giác máy tính.

Deep networks thực tế rất kém trong việc học hàm đồng nhất. Do đó, việc truyền trực tiếp các embedding "thô" tới lớp tiếp theo thực sự có thể rất hữu ích!

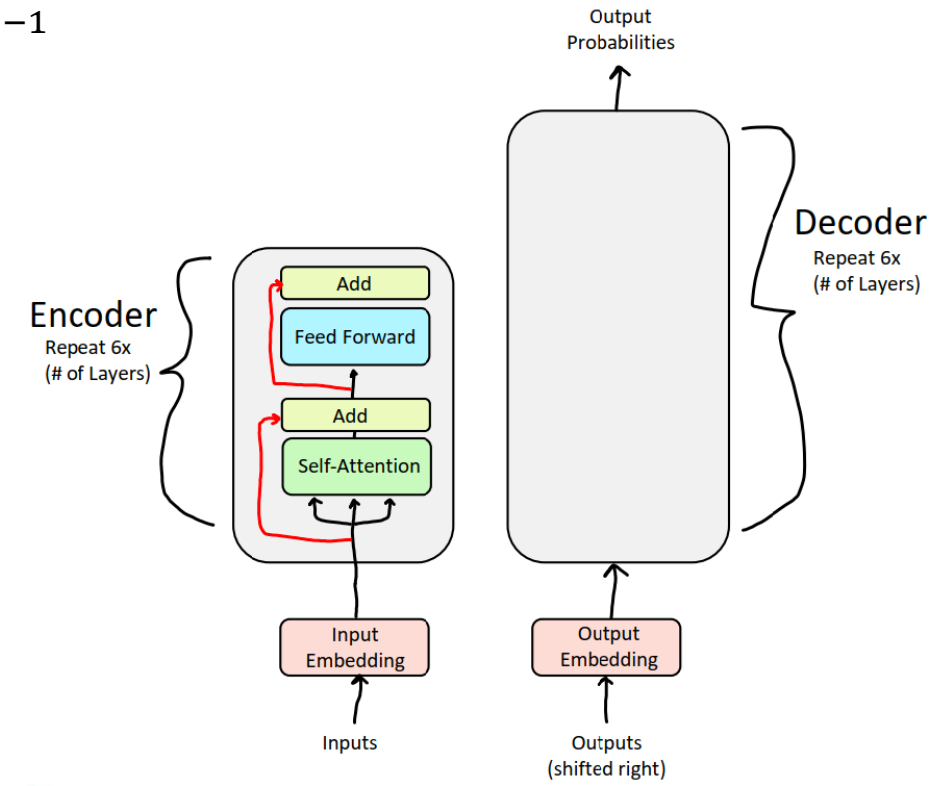
$$x_l = F(x_{l-1}) + x_{l-1}$$

- Điều này ngăn mạng "quên" hoặc làm biến dạng các thông tin quan trọng khi nó đi qua nhiều lớp.
- Đồng thời làm mịn bề mặt hàm mất mát và giúp huấn luyện dễ dàng hơn.

Residual connections are also thought to smooth the loss landscape and make training easier!



[Loss landscape visualization, Li et al., 2018, on a ResNet]



Training Trick #2: Layer Normalization

Vấn đề: Khó huấn luyện các tham số của một lớp nhất định vì đầu vào của nó từ lớp bên dưới liên tục thay đổi.

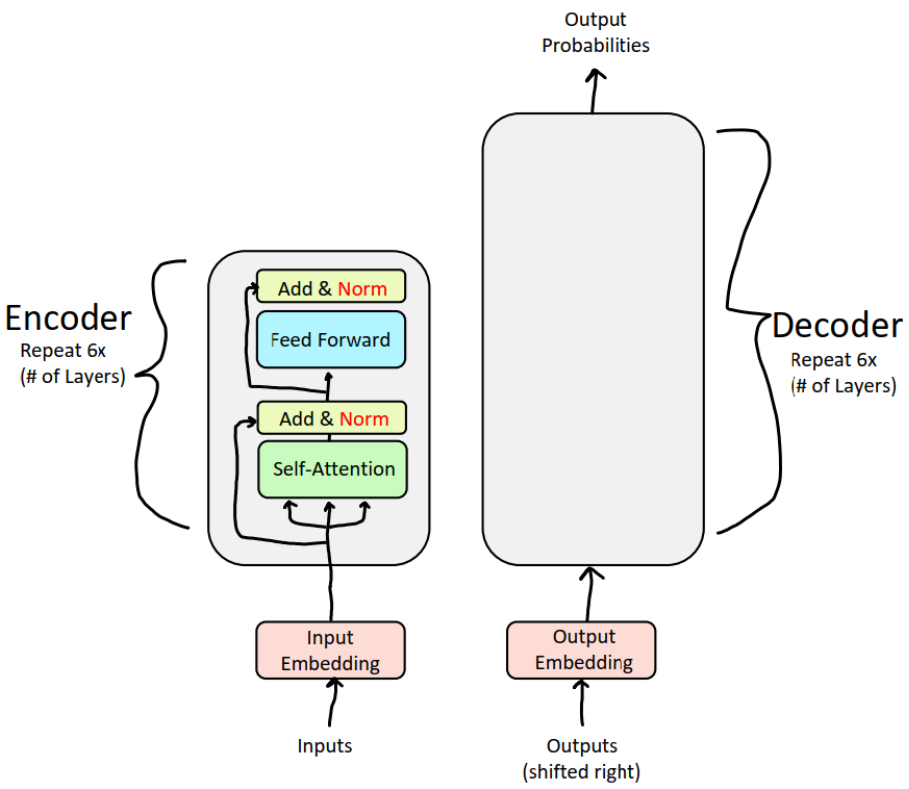
Giải pháp: Giảm thiểu sự biến thiên bằng cách chuẩn hóa về giá trị trung bình bằng 0 và độ lệch chuẩn bằng 1 trong mỗi lớp.

Được tính toán trên các đặc trưng một cách độc lập cho mỗi mẫu.

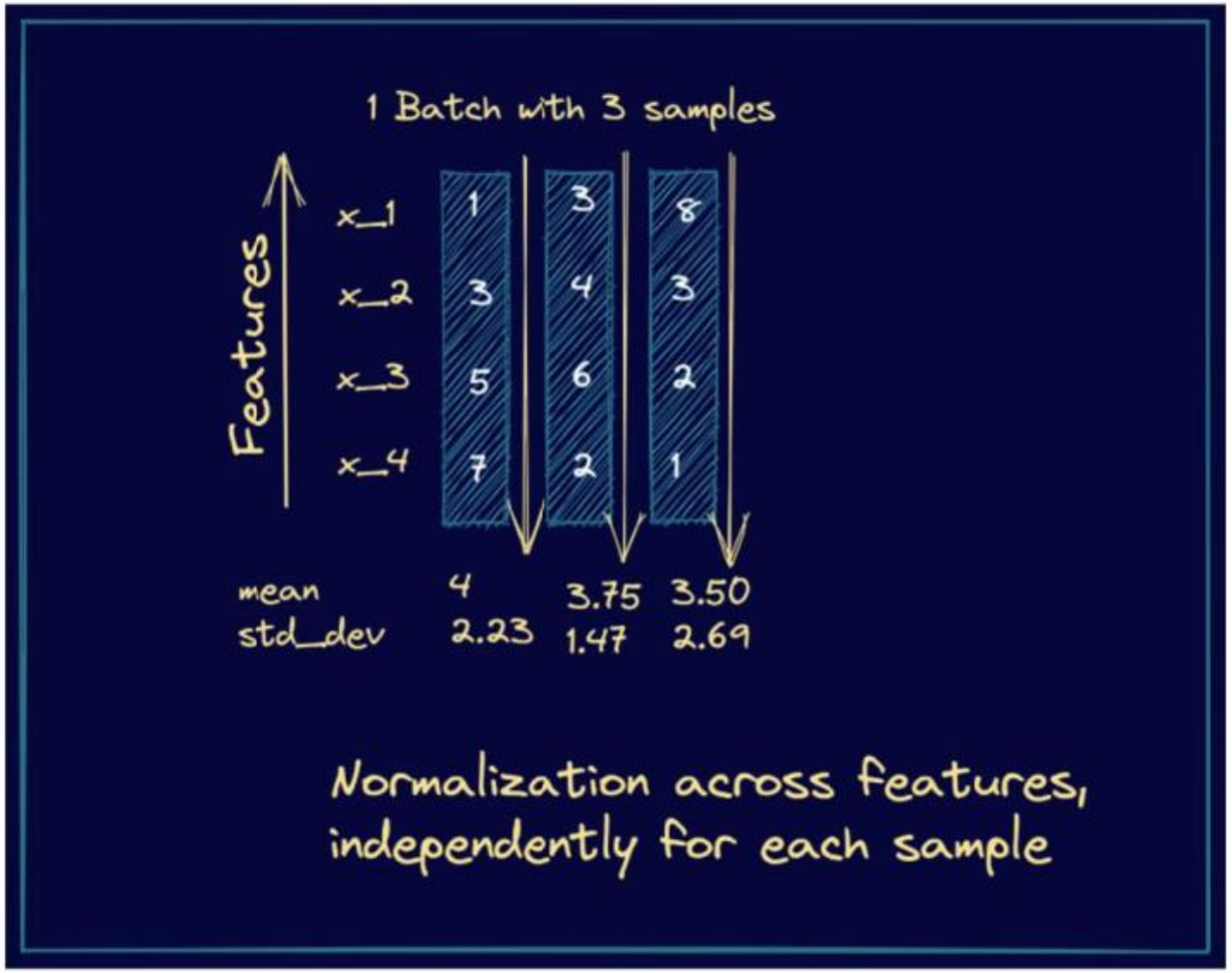
$$\text{Mean: } \mu = \frac{1}{H}\sum_{i=1}^{H} a_i$$

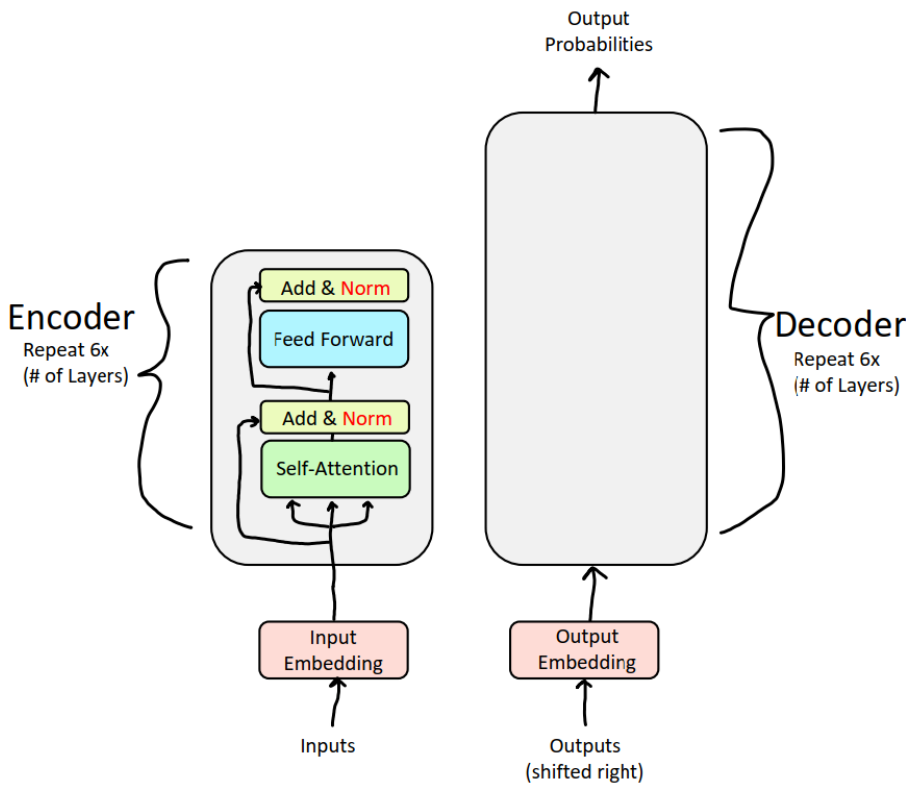
$$\text{Standard deviation: } \sigma = \sqrt{\frac{1}{H}\sum_{i=1}^{H}(a_i - \mu)}$$

$$x^{l'} = \frac{x^l - \mu}{\sigma + \varepsilon}$$



Training Trick #2: Layer Normalization





Training Trick #3: Scaled Dot Product Attention

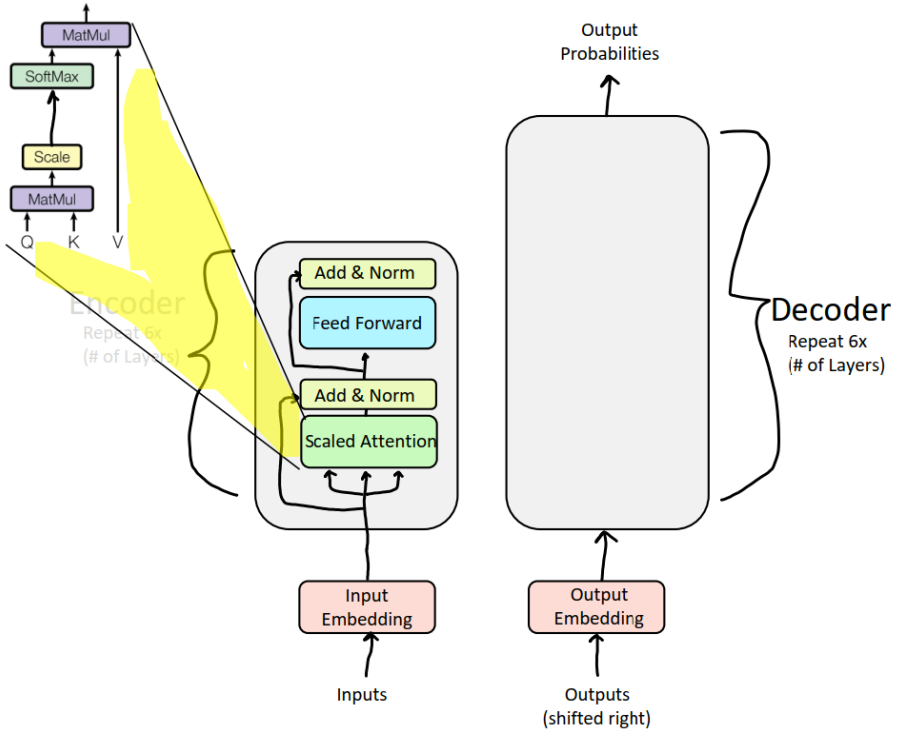
Sau LayerNorm, giá trị trung bình và phương sai của các phần tử vector lần lượt là 0 và 1. Tuy nhiên, tích vô hướng vẫn có xu hướng nhận các giá trị cực đoan, vì phương sai của nó thay đổi theo số chiều d_k

Quick Statistics Review:

- Mean of sum = sum of means = $d_k * 0 = 0$
- Variance of sum = sum of variances = $d_k * 1 = d_k$
- To set the variance to 1, simply divide by $\sqrt{d_k}$!

Updated Self-Attention Equation:

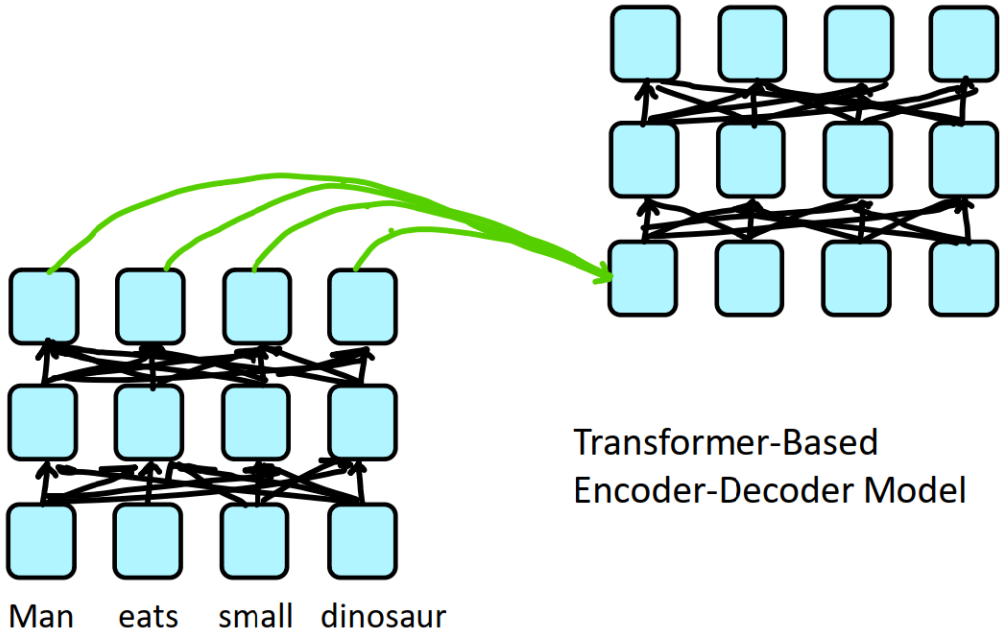
$$Output = softmax\left(QK^T / \sqrt{d_k}\right) V$$



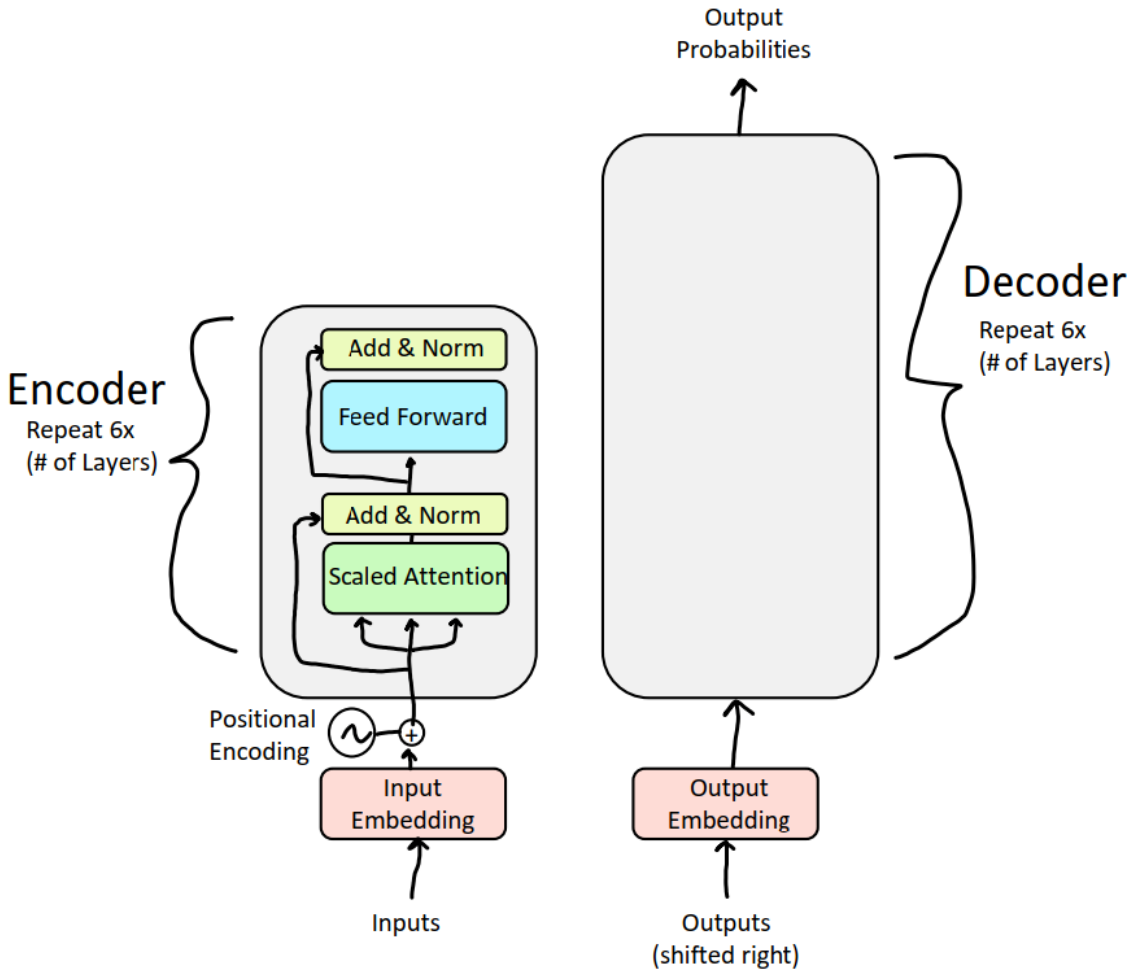
Vấn đề lớn!

- Chúng ta gần như đã hoàn thành với Encoder, nhưng có một vấn đề lớn!
- Thứ tự không tác động gì đến mạng cả!
- Điều này có vẻ sai vì thứ tự của từ có mang ý nghĩa trong nhiều ngôn ngữ, bao gồm cả tiếng Anh.

$$Output = softmax\left(QK^T / \sqrt{d_k}\right)V$$



Giải pháp:
Đưa thông tin thứ tự vào, thông qua Positional Encodings



Khắc phục vấn đề self-attention: thứ tự chuỗi

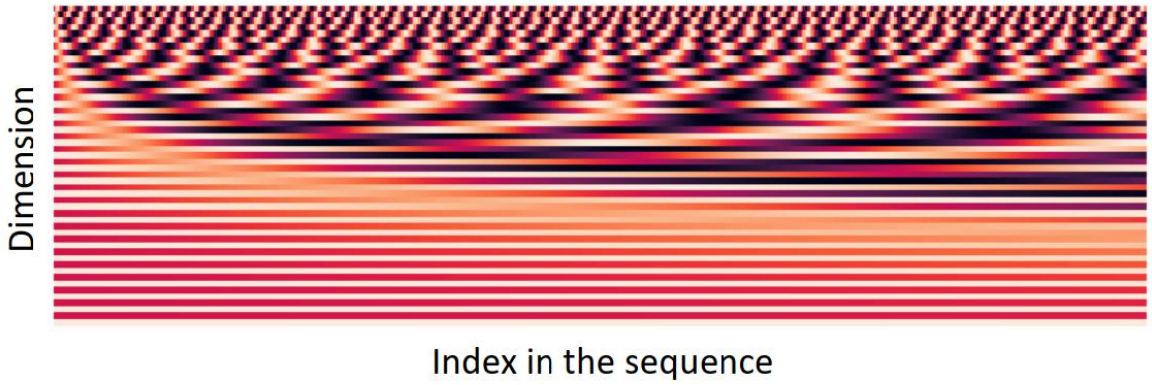
- Do tự chú ý không xây dựng sẵn thông tin thứ tự, chúng ta cần mã hóa thứ tự của câu vào các keys ($\tilde{k}_i$), queries ($\tilde{q}_i$) và values ($\tilde{v}_i$).
- Xem xét việc biểu diễn mỗi chỉ số chuỗi như một vector vị trí $p_i \in R^d$, với $i \in \{1,2, \dots , T\}$
- Cộng p_i vào đầu vào:

$$v_i = \tilde{v}_i + p_i$$
$$q_i = \tilde{q}_i + p_i$$
$$k_i = \tilde{k}_i + p_i$$

Các vector biểu diễn vị trí thông qua các hàm sin

Cơ chế: Ghép nối (concatenate) các hàm hình sin và cosin với các chu kỳ khác nhau để tạo ra vector biểu diễn vị trí:

$$p_i = \begin{pmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{pmatrix}$$

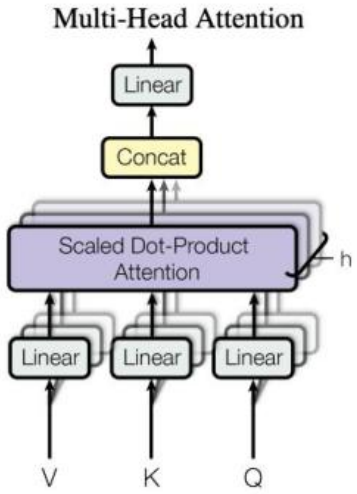


Các vector biểu diễn vị trí thông qua các hàm sin

- **Ưu điểm:**
  - Tính chu kỳ: Thể hiện rằng "vị trí tuyệt đối" có thể không quan trọng bằng mối quan hệ tương đối giữa các từ.
  - Khả năng ngoại suy (Extrapolation): Về lý thuyết, mô hình có thể xử lý các chuỗi dài hơn độ dài trong tập huấn luyện nhờ vào việc các chu kỳ được lặp lại.
- **Nhược điểm:**
  - Không thể học (Not learnable): Các tham số được cố định sẵn, mô hình không thể điều chỉnh bộ mã hóa này trong quá trình huấn luyện.
  - Hiệu quả thực tế: Việc ngoại suy cho các chuỗi dài hơn, thực tế không hoạt động hiệu quả như kỳ vọng ban đầu.

Multi-Headed Self-Attention: k heads tốt hơn 1!

Ý tưởng: Hãy thực hiện sự tự chú ý nhiều lần song song và kết hợp các kết quả lại. (Tương tự như các kernel trong ConvNet, mỗi đầu chú ý đến một thông tin khác nhau).



[Vaswani et al. 2017]

The Transformer Encoder: Multi-headed Self-Attention

- Nếu chúng ta muốn nhìn vào nhiều vị trí trong câu cùng một lúc thì sao?
- Chúng ta sẽ xác định **multiple attention heads** thông qua nhiều ma trận Q, K, V.
- Với $Q_l, K_l, V_l \in R^{d \times \frac{d}{h}}$, trong đó h là số attention heads và l nằm trong khoảng 1 tới h.
- Mỗi attention heads thực hiện chú ý độc lập:

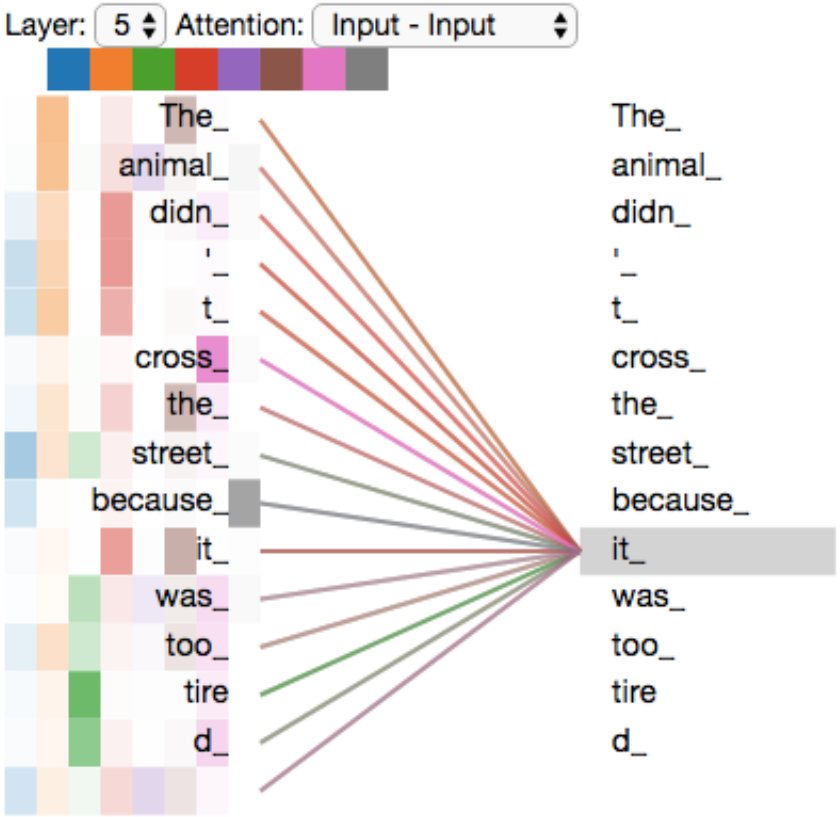
$$output_l = softmax\left(XQ_lK_l^TX^T\right) * XV_l, \text{ trong đó } output_l \in R^{d/h}$$

- Sau đó, đầu ra của tất cả các attention heads được kết hợp lại:

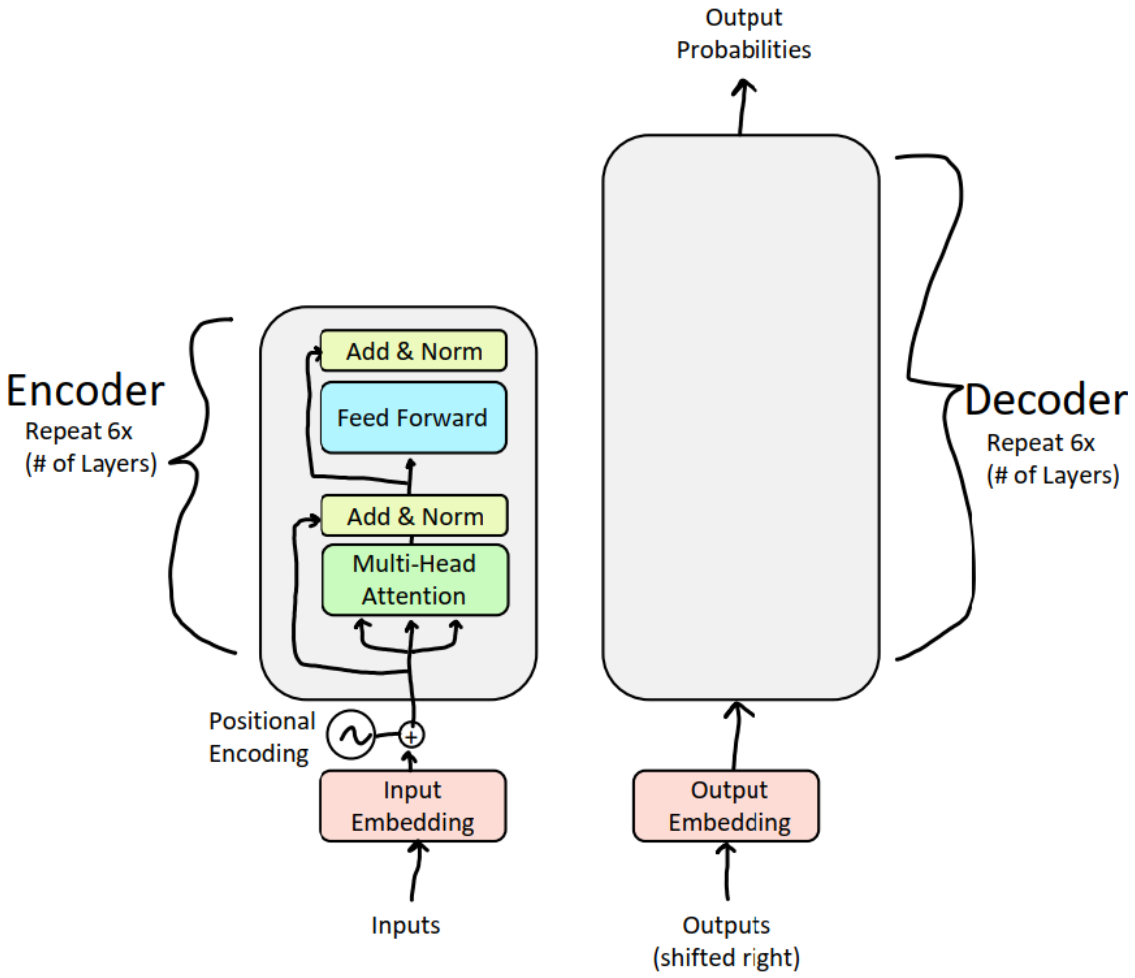
$$output = Y[output_1; \dots; output_h], \text{ trong đó } Y \in R^{d \times d}$$

- Mỗi head được "nhìn" vào các thứ khác nhau và xây dựng các vector giá trị theo những cách khác nhau.

Multi-headed self-attention



Transformer Decoder



Decoder: Masked Multi-Head Self-Attention

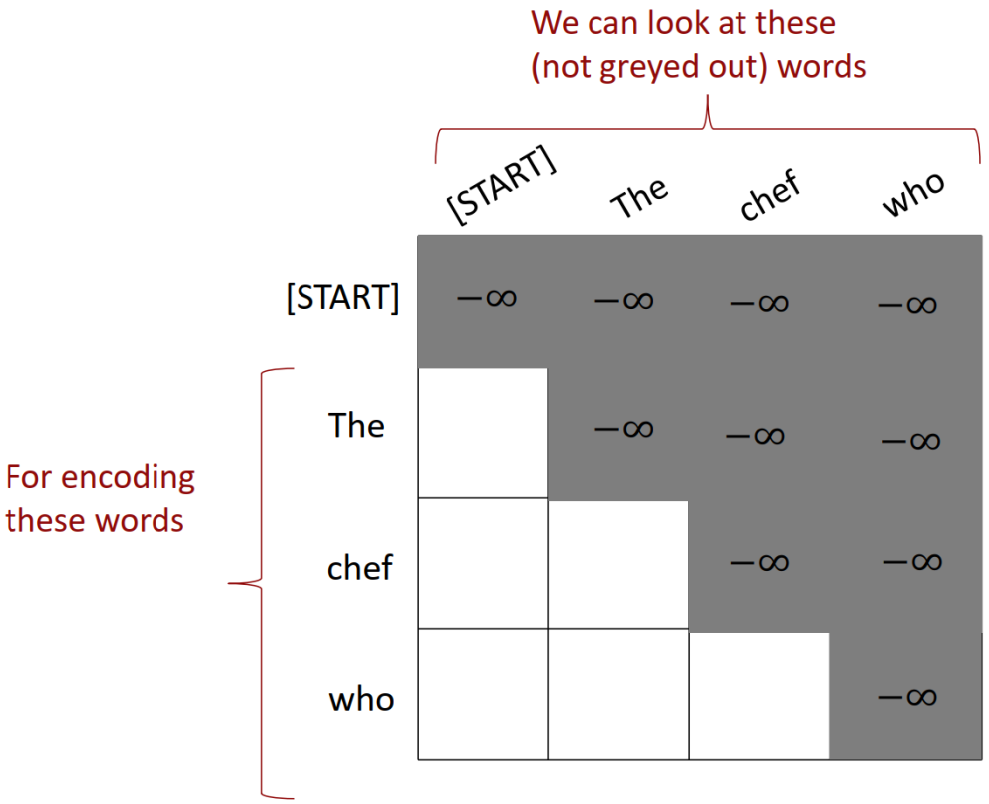
Vấn đề: Nếu chúng ta có mục tiêu mô hình hóa ngôn ngữ, chẳng phải mạng chỉ cần nhìn về phía trước và "thấy" câu trả lời sao? Làm thế nào để giữ cho decoder không "gian lận"?

Giải pháp: Sử dụng Masked Multi-Head Attention. Về cơ bản, chúng ta ẩn (che giấu) thông tin về các token tương lai khỏi mô hình.

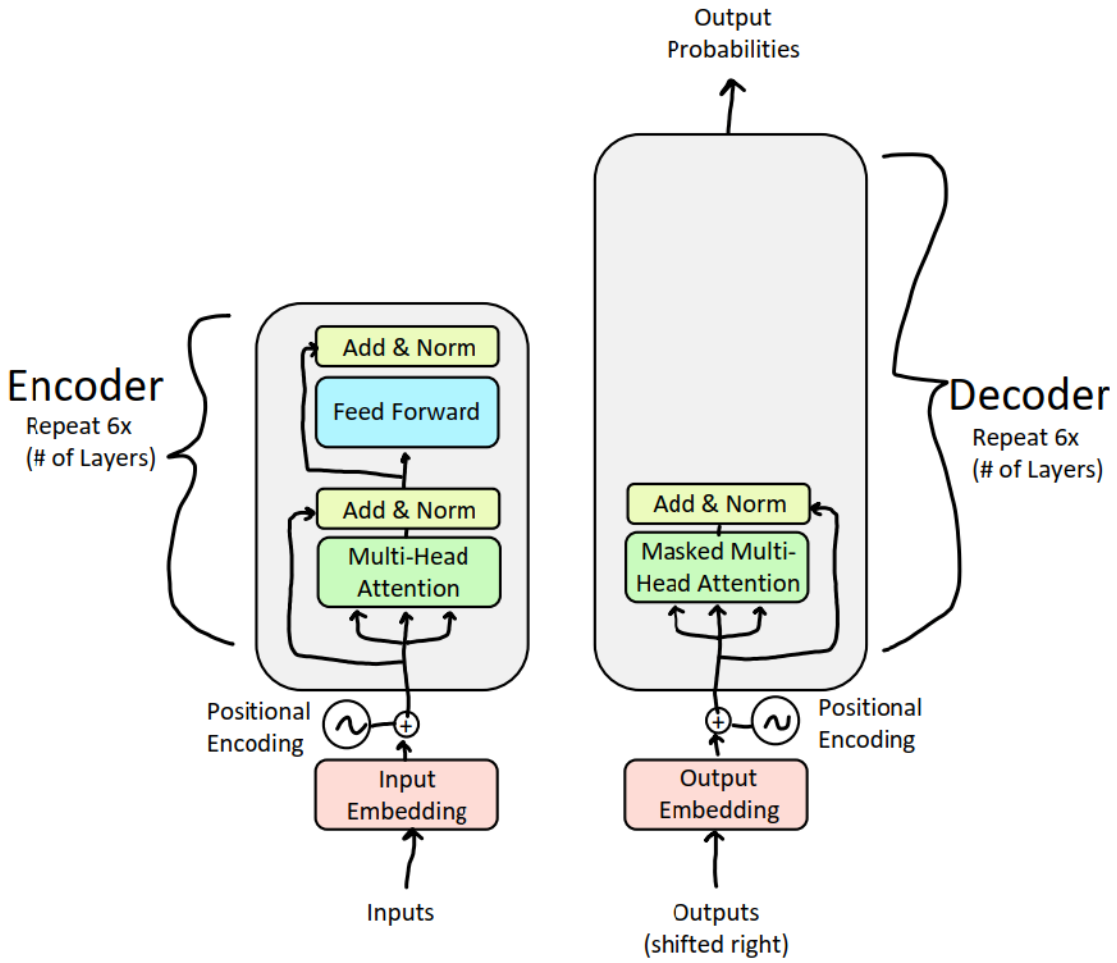
Masking the future in self-attention

Để cho phép song song hóa, chúng ta che đi sự chú ý vào các từ trong tương lai bằng cách đặt điểm chú ý (attention scores) thành $-\infty$ đối với $j \geq i$.

$$e_{ij} = \begin{cases} q_i^T k_j, j < i \\ -\infty, j \geq i \end{cases}$$



Decoder: Masked Multi-Headed Self-Attention



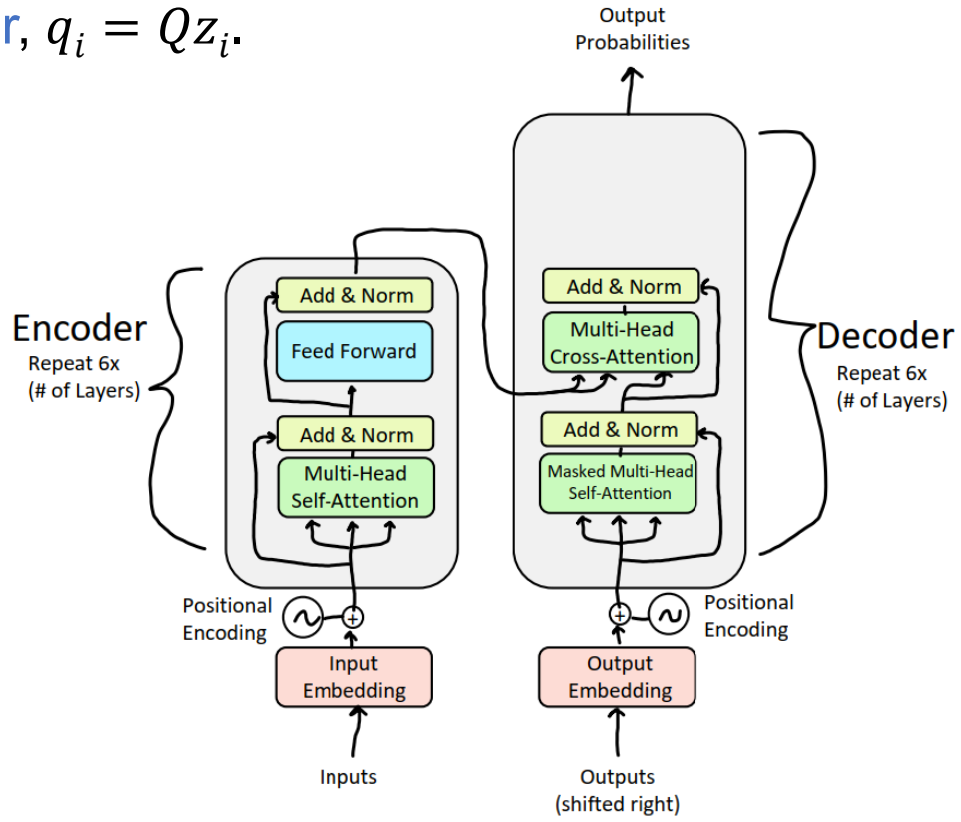
Encoder-Decoder Attention

Đặt $h_1, \dots, h_T$ là output vectors từ Transformer encoder; $h_i \in R^d$

$z_1, \dots, z_T$ là input vectors từ Transformer decoder, $z_i \in R^d$

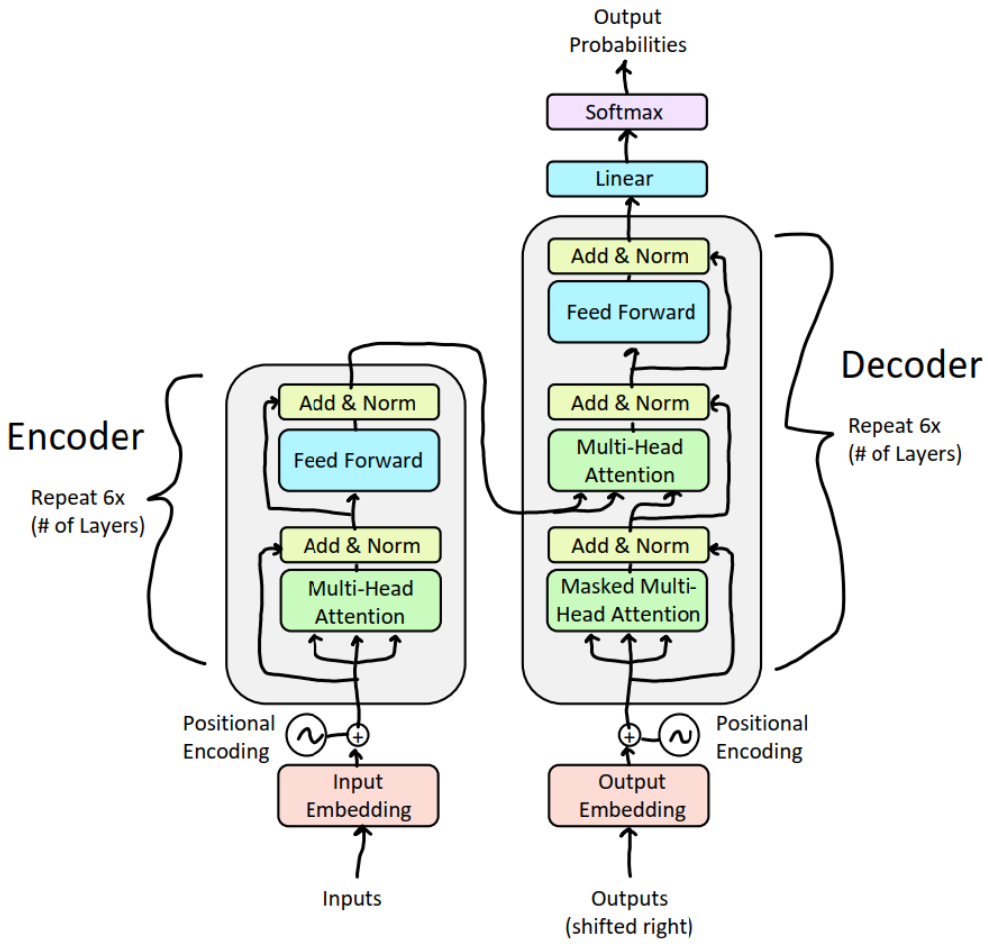
Khi đó keys và values được rút ra từ encoder (giống một bộ nhớ): $k_i = Khi, vi = Vh_i$

Và các queries được rút ra từ decoder, $q_i = Qz_i$.



Decoder: Hoàn thiện!

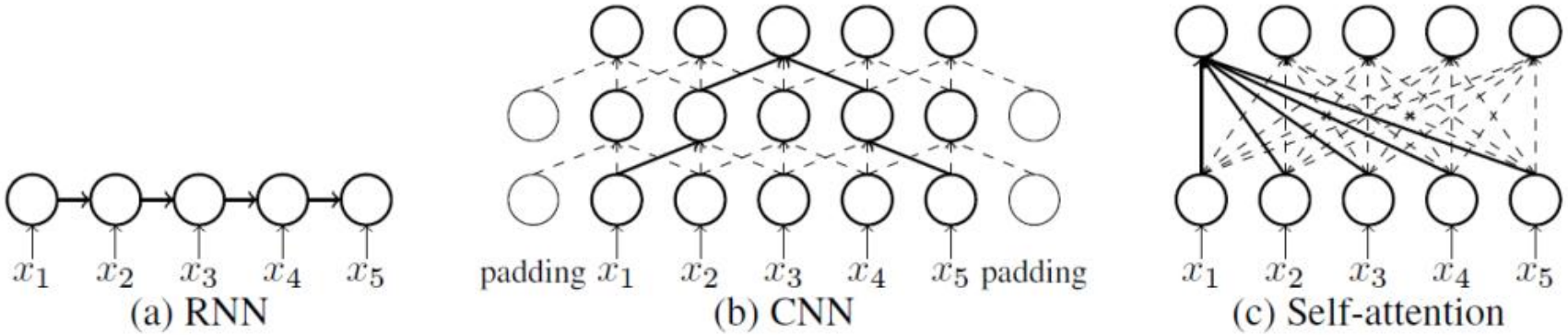
- Thêm một lớp feed-forward (có các kết nối phần dư và layer norm).
- Thêm một lớp tuyến tính cuối cùng để chiếu các embedding thành một vector dài hơn nhiều, có kích thước bằng từ vựng (logits).
- Thêm một hàm softmax cuối cùng để tạo ra phân phối xác suất của các từ tiếp theo có thể xảy ra.



Nhược điểm và các biến thể của Transformer

Ưu điểm

- Không có tính hồi quy (No recurrence): cho phép mã hóa song song.
- Huấn luyện nhanh: cả encoder và decoder đều chạy song song.
- Không gặp vấn đề khoảng cách xa (No long-range problem): Mọi token đều có kết nối trực tiếp với độ phức tạp O(1).
- Ba loại chú ý: mô hình không phải nhớ quá nhiều thứ.
- Sự chú ý đa đầu (Multi-head attention) cho phép chú ý đến nhiều khía cạnh khác nhau



Chúng ta muốn sửa chữa điều gì ở Transformer?

- Chi phí tính toán bình phương trong self-attention
  - Độ phức tạp $O(n^2)$: Việc tính toán sự tương tác giữa tất cả các cặp từ đồng nghĩa với việc khối lượng tính toán sẽ tăng theo hàm bình phương so với độ dài chuỗi $O(n)$.
  - So sánh với RNN: Trong các mô hình tuần tự (Recurrent models), khối lượng tính toán chỉ tăng theo hàm tuyến tính $O(n)$.
- Hệ quả: Đây là "điểm nghẽn" khiến Transformer khó xử lý các văn bản cực dài nếu không có các kỹ thuật tối ưu hóa bộ nhớ.

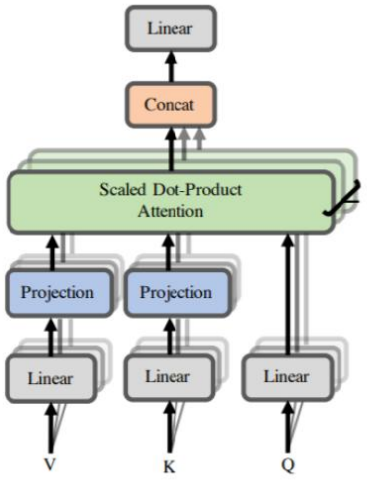
Chúng ta muốn sửa chữa điều gì ở Transformer?

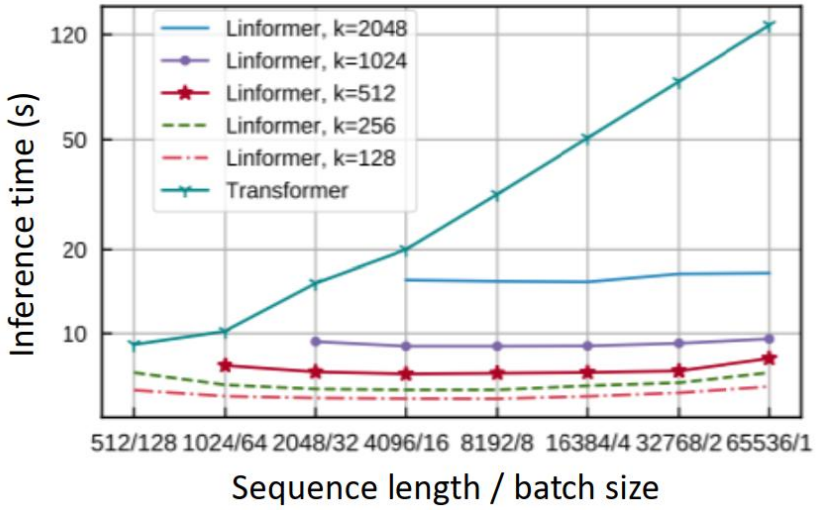
- Các phương pháp biểu diễn vị trí tiên tiến: Liệu việc sử dụng các chỉ số tuyệt đối đơn giản (absolute indices) đã là cách tốt nhất để biểu diễn vị trí?
- Các nghiên cứu mới đã đề xuất:
  - Relative linear position attention [Shaw et al., 2018]: Tập trung vào khoảng cách tương đối giữa các từ thay vì vị trí tuyệt đối trong câu.
  - Dependency syntax-based position [Wang et al., 2019]: Tận dụng cấu trúc ngữ pháp và mối quan hệ giữa các thành phần trong câu để xác định vị trí.
  - Rotary Embeddings (RoPE) [Su et al., 2021]: Sử dụng ma trận quay để tích hợp thông tin vị trí vào phép toán Dot-product. (Đây là phương pháp cốt lõi đang được sử dụng trong các mô hình ngôn ngữ lớn hiện đại như Llama hay Mistral.)

Nghiên cứu gần đây về cải thiện chi phí tự chú ý bậc hai

Có rất nhiều công trình gần đây giải quyết câu hỏi: Liệu chúng ta có thể xây dựng các mô hình giống như Transformers mà không phải trả chi phí all-pairs self-attention $O(T^2)$?
Ví dụ, Linformer [Wang et al., 2020]

Key idea: map the sequence length dimension to a lower-dimensional space for values, keys

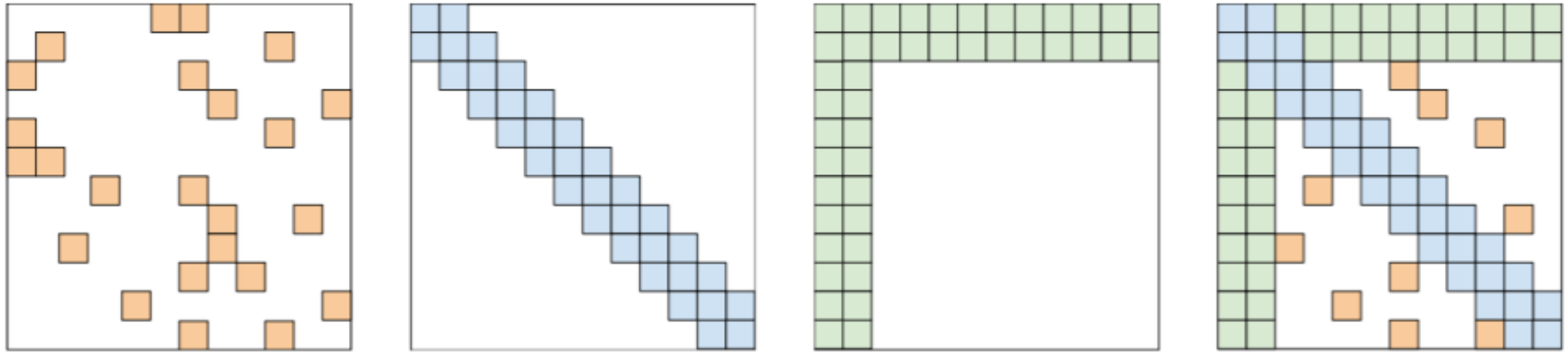




Nghiên cứu gần đây về cải thiện chi phí tự chú ý bậc hai

BigBird [Zaheer et al., 2021]

- Ý tưởng chính là thay thế các tương tác all-pairs interactions bằng một họ các tương tác khác, như window attention, global attention, và random attention



(a) Random attention (b) Window attention (c) Global Attention (d) BIGBIRD